

MLP Exam Prep

Concept Mastery Guide — Part 1 of 4

Pandas Foundations & Data Preprocessing

This guide is designed for the IIT Madras BS Diploma **Machine Learning Practice** course. Every concept is explained in plain language with the assumption that you have **weak Pandas** and limited ML background. The focus is on what shows up in your exam: practical sklearn usage, code-output prediction, and parameter understanding.

What's in Part 1

#	Section	Why it matters for the exam
1	How to read Python code (refresher)	Every PYQ is code-based
2	NumPy in 5 minutes	Underlies every sklearn method
3	Pandas Series & DataFrame	Datatype questions appear in every paper
4	Data selection: loc, iloc, boolean filtering	Asked directly & indirectly
5	Dropping, applying, statistics in Pandas	Common in code snippets
6	Intro to scikit-learn architecture	Bunch object, fit/transform pattern
7	Loading data: load_* vs fetch_*	Asked in EVERY PYQ paper
8	Handling missing data (imputation)	L2.4.2 — common
9	Categorical encoding	L2.5 — common
10	Numeric transformers & feature scaling	L2.6 — heavy weightage
11	Outlier detection & removal	L2.6.3 — common

1. How to Read Python Code (Quick Refresher)

Before we touch any ML topic, you must be able to look at a 5-line Python snippet and predict what it does. Almost every exam question shows code and asks for the output. Here are the patterns you'll see repeatedly.

Imports — what library is being used?

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
```

When you see **from sklearn.X import Y**, that tells you what kind of problem we're dealing with. **preprocessing** = data cleaning. **linear_model** = regression or logistic regression. **tree** = decision trees. **ensemble** = RandomForest, boosting. **cluster** = KMeans, HAC. **metrics** = scoring (accuracy, f1, etc.).

The fit / transform / predict pattern

Almost every sklearn class follows this 3-step rhythm:

- **fit(X, y)** — "learn" from training data. Computes means, weights, splits, etc. Stores them inside the object.
- **transform(X)** — apply what was learned to data (used for preprocessors like scalers, encoders).
- **predict(X)** — apply what was learned to predict labels/values (used for models like LinearRegression).
- **fit_transform(X)** — shortcut that does fit then transform. Use only on training data.

WATCH OUT: The most common exam trap: Calling **fit_transform** on test data is WRONG. On test data you must call **transform** only — otherwise the test data "leaks" into the learned parameters, which is called data leakage. Same rule for the target variable: you fit a model on `y_train`, never on `y_test`.

Reading shape and dimensions

When code does **X.shape**, the answer is a tuple (**rows, columns**). For a 1-D array (a vector), you get just (**n,**) with one number. This shows up in loaders/fetchers questions a lot.

```
X = np.array([[1, 2, 3], [4, 5, 6]])
X.shape    # (2, 3) — 2 rows, 3 columns

y = np.array([1, 2, 3, 4, 5])
y.shape    # (5,) — 1-D vector with 5 elements
```

2. NumPy in 5 Minutes

NumPy is the layer underneath everything. Every dataset in sklearn is internally a NumPy array. You don't need to be an expert, but you must recognize these:

Creating arrays

```
np.array([1, 2, 3])           # 1-D array, shape (3,)
np.array([[1, 2], [3, 4]])    # 2-D array, shape (2, 2)
np.zeros((3, 4))              # 3x4 array of zeros
np.ones((2, 5))               # 2x5 array of ones
np.arange(10)                 # [0, 1, 2, ..., 9]
np.linspace(0, 1, 5)          # [0.0, 0.25, 0.5, 0.75, 1.0]
```

Indexing and slicing

```
a = np.array([[10, 20, 30],
               [40, 50, 60],
               [70, 80, 90]])

a[0]           # first row: [10, 20, 30]
a[:, 0]        # first column: [10, 40, 70]
a[1, 2]        # row 1, col 2: 60
a[0:2, 1:]     # first 2 rows, cols from 1 onwards: [[20, 30], [50, 60]]
```

NOTE: Slicing rule: start:stop — start is included, stop is NOT. So **a[0:2]** gives rows 0 and 1, not 2.

Reshaping

reshape(rows, cols) changes the shape without changing the data. A common need is converting a 1-D array of n elements into a column vector of shape (n, 1) before passing it to sklearn's fit method, because sklearn expects 2-D input for X.

```
x = np.array([1, 2, 3, 4])
x.reshape(-1, 1)
# array([[1],
#        [2],
#        [3],
#        [4]])
# shape is now (4, 1)
# The -1 means "figure it out automatically"
```

Useful array operations that show up in exams

Method	What it does	Example output
a.mean()	Mean of all elements (or along axis)	a.mean() → scalar
a.std()	Standard deviation	a.std() → scalar
a.sum(axis=0)	Sum along columns (collapse rows)	shape (3,) for 2x3 input
a.sum(axis=1)	Sum along rows (collapse columns)	shape (2,) for 2x3 input
np.argmax(a)	Index of maximum element	integer
np.argmin(a)	Index of minimum element	integer
a.T	Transpose: rows ↔ columns	shape (m,n) → (n,m)
np.dot(a, b) or a @ b	Matrix multiplication	depends on shapes

TIP: **axis = 0** means "down the rows, collapse them, give one answer per column." **axis = 1** means "across the columns, collapse them, give one answer per row." Memorize this — it's asked directly in some PYQs.

3. Pandas Series & DataFrame

Pandas is the workhorse for tabular data. There are two main objects: **Series** (1-D, like a single column with labels) and **DataFrame** (2-D, like a spreadsheet with named columns and indexed rows). The IIT-M PYQ asks you to identify which one you're looking at — get this right.

3.1 Series — a labelled 1-D array

```
import pandas as pd

s = pd.Series([10, 20, 30, 40], index=['a', 'b', 'c', 'd'])
print(s)
# a      10
# b      20
# c      30
# d      40
# dtype: int64

s['b']      # 20 (label-based access)
s[1]        # 20 (position-based – works but discouraged in newer pandas)
s.values    # array([10, 20, 30, 40])
s.index     # Index(['a', 'b', 'c', 'd'])
```

A Series is essentially a NumPy array with a label for each element. Most things you can do to a NumPy array (mean, sum, math operations) work on a Series too.

3.2 DataFrame — a labelled 2-D table

```
df = pd.DataFrame({
    'name': ['Alice', 'Bob', 'Charlie'],
    'age':  [25, 30, 35],
    'city': ['Mumbai', 'Delhi', 'Pune']
})
print(df)
#      name  age  city
# 0   Alice   25  Mumbai
# 1    Bob    30   Delhi
# 2  Charlie   35    Pune
```

A DataFrame is a dictionary of Series, where each Series is a column. Every column has a name (the dictionary key). Every row has an index (default 0, 1, 2, ...).

Inspecting a DataFrame — the methods you'll see in exam code

Method / attribute	What it returns	Why exam asks
df.shape	(rows, columns) tuple	Always asked
df.head(n)	First n rows (default 5)	Common
df.tail(n)	Last n rows (default 5)	Common
df.info()	Datatypes, non-null counts, memory	Less common
df.describe()	Stats (count, mean, std, min, quartiles, max) for numeric columns	Common
df.dtypes	Datatype of each column	Common
df.columns	List of column names	Common
df.index	Row labels	Less common
df.values	Underlying NumPy array	Common
df.isnull().sum()	Count of missing values per column	Asked in imputation Qs

NOTE: Datatype identification questions: The exam will show code that loads something and ask you to pick: 'bunch', 'dataframe', 'series', 'array'. Remember: **load_iris()** returns a **Bunch**. **load_iris(as_frame=True)** returns a Bunch where **.data** is a DataFrame. Single column extracted from a DataFrame, e.g. **df['age']**, is a **Series**. Two-column slice **df[['age', 'name']]** is still a **DataFrame**.

4. Selecting Data: loc, iloc, Boolean Filtering

This is the #1 source of confusion for beginners and a frequent topic in exam code. There are three ways to grab parts of a DataFrame:

4.1 Bracket access — quickest

```
df['age']          # one column, returns a Series
df[['age', 'city']] # multiple columns, returns a DataFrame
df[0:2]           # rows 0 and 1 (slicing, NOT single index!)
```

WATCH OUT: Trap: `df[0]` does NOT give you row 0. Pandas thinks you're asking for a column named 0, which doesn't exist — `KeyError`. To get rows by position, use **iloc**.

4.2 .loc — label-based access

.loc accesses rows and columns by their **labels** (index values, column names). Format: **df.loc[row_label, col_label]**.

```
df.loc[0, 'age']          # value at row label 0, column 'age' → 25
df.loc[0:2, 'name':'city'] # rows 0,1,2 ; columns from 'name' to 'city'
df.loc[:, 'age']          # all rows, just 'age' column
df.loc[df['age'] > 28]     # rows where age > 28 (boolean filter)
```

NOTE: Critical .loc rule: with **.loc**, slicing is **INCLUSIVE on both ends**. `df.loc[0:2]` gives rows 0, 1, AND 2 (three rows). This differs from regular Python slicing!

4.3 .iloc — position-based access

.iloc accesses rows and columns by their integer **position** (0, 1, 2, ...) regardless of what the labels say.

```
df.iloc[0, 1]          # row position 0, column position 1 → 25 (age)
df.iloc[0:2, 0:2]      # first 2 rows, first 2 columns
df.iloc[-1]            # last row
df.iloc[[0, 2]]        # rows at positions 0 and 2
```

NOTE: Critical .iloc rule: with **.iloc**, slicing is **EXCLUSIVE on the end** (like normal Python). `df.iloc[0:2]` gives rows 0 and 1 only — NOT row 2.

4.4 Boolean filtering

You can filter rows by passing a boolean Series of the same length:

```
mask = df['age'] > 28          # Series of True/False values
df[mask]                       # only rows where mask is True
```

```
# Combine conditions with & (and), | (or), ~ (not):
df[(df['age'] > 25) & (df['city'] == 'Pune')]
df[~df['name'].isin(['Alice', 'Bob'])] # everyone except Alice and Bob
```

WATCH OUT: Use & and | (NOT 'and' / 'or'): Pandas requires the bitwise operators because it operates element-wise on Series. Wrap each condition in parentheses, otherwise Python's operator precedence will trip you up.

Comparison summary

Method	Selection by	End-inclusive slicing?	in	Typical use
df['col']	Column name	—		Get one column
df.loc[r, c]	Labels (index, col name)	YES		Most readable
df.iloc[r, c]	Integer positions	NO (Python rule)		When you need by position
df[boolean_mask]	Row condition	—		Filter rows

5. Dropping, Applying, and Statistics in Pandas

5.1 drop() — removing rows or columns

```
df.drop('age', axis=1)          # drop the 'age' column (axis=1 = columns)
df.drop(['age', 'city'], axis=1) # drop multiple columns
df.drop(0, axis=0)              # drop row with index 0 (axis=0 = rows)

# IMPORTANT: drop() returns a NEW DataFrame by default. The original is unchanged.
# To modify in place:
df.drop('age', axis=1, inplace=True)
```

NOTE: axis=0 vs axis=1: axis=0 means "go down the rows" (so you're acting on rows). axis=1 means "go across the columns" (so you're acting on columns). This trips up many students. **drop with axis=1 drops a COLUMN.**

5.2 apply() — running a function on a Series or DataFrame

```
# Apply on a Series (one column)
df['age_doubled'] = df['age'].apply(lambda x: x * 2)

# Apply on a DataFrame, axis=0 (default): function gets each COLUMN as input
df.apply(np.mean, axis=0)

# Apply on a DataFrame, axis=1: function gets each ROW as input
df.apply(lambda row: row['age'] + 5, axis=1)
```

5.3 Statistics on a DataFrame

These all skip non-numeric columns automatically and ignore NaN by default:

Method	Returns	Notes
df.mean()	Series of column means	axis=0 default
df.median()	Series of column medians	
df.std()	Standard deviation	uses sample std (N-1) by default
df.var()	Variance	
df.min(), df.max()	Min / Max per column	
df.sum()	Sum per column	
df.count()	Non-null count per column	Useful for missing data
df.describe()	All of above in one table	For numeric cols only by default
df.corr()	Correlation matrix	Pearson by default; method='spearman' available
df.value_counts()	Frequency of each unique value	On a Series

5.4 GroupBy — split / apply / combine

Briefly, because exams sometimes touch this:

```
df.groupby('city')['age'].mean()  
# Returns mean age for each city.  
# This is split → apply → combine: split by city, apply mean, combine into a Series.
```

6. Introduction to Scikit-Learn

Scikit-learn (imported as **sklearn**) is the main ML library you'll be tested on. Its design has a few principles that, once you understand them, make every PYQ easier.

6.1 The Estimator API

Every algorithm in sklearn is implemented as an **estimator** — a Python class with a consistent interface. There are three main flavors:

- **Transformers:** objects that transform data. They have **fit()** and **transform()**. Examples: StandardScaler, OneHotEncoder, PCA, SimpleImputer.
- **Predictors:** objects that make predictions. They have **fit()** and **predict()**. Examples: LinearRegression, LogisticRegression, DecisionTreeClassifier, KNeighborsClassifier.
- **Meta-estimators:** wrap other estimators. Examples: GridSearchCV, Pipeline, BaggingClassifier.

6.2 The fit / transform / predict pattern (in depth)

```
# Step 1: instantiate the estimator (set hyperparameters)
scaler = StandardScaler()

# Step 2: fit on training data (learn the mean and std)
scaler.fit(X_train)

# Step 3a: transform training data (apply learned mean/std)
X_train_scaled = scaler.transform(X_train)

# Step 3b: transform test data (apply SAME learned mean/std – don't refit!)
X_test_scaled = scaler.transform(X_test)

# Or combine fit + transform on training (shortcut):
X_train_scaled = scaler.fit_transform(X_train)
```

WATCH OUT: Why we never fit_transform on test data: If you do, the scaler computes a NEW mean and std using the test data, which is information your model wasn't supposed to see during training. This is called **data leakage** and inflates your test scores artificially. Exam loves this question.

6.3 The Bunch object — sklearn's mini-dictionary

When you call **load_iris()**, **load_digits()**, **fetch_california_housing()**, or any of sklearn's data-loading functions, you don't get back a DataFrame. You get a **Bunch** object. A Bunch is a dictionary that **also** lets you access keys as attributes (with a dot).

```
from sklearn.datasets import load_iris

iris = load_iris()
type(iris)          # <class 'sklearn.utils.Bunch'>

# Access via dictionary-style key access:
iris['data']         # the feature matrix X
iris['target']       # the target labels y
iris['feature_names'] # column names
```

```
iris['target_names']    # class names

# Or attribute-style access:
iris.data
iris.target
iris.feature_names
iris.DESCR              # long description string
```

NOTE: Asked every paper: 'What does `load_iris()` return?' Answer: a **Bunch** object. It contains **data** (the X matrix as a NumPy array), **target** (the y vector), and metadata. To get a DataFrame instead, pass **as_frame=True**: `load_iris(as_frame=True).data` is then a DataFrame.

7. Loading Data: load_* vs fetch_*

This is one of the most reliable exam questions. Sklearn ships with two families of data-loading functions, and the difference matters:

	load_*	fetch_*
Where data lives	Bundled with sklearn — already on your machine	Downloaded from internet on first use, cached locally
Size	Small toy datasets (KB scale)	Larger real-world datasets (MB scale)
Examples	load_iris, load_digits, load_wine, load_breast_cancer, load_diabetes	fetch_california_housing, fetch_openml, fetch_20newsgroups, fetch_olivetti_faces
Returns	Bunch object	Bunch object
Typical use	Quick examples, classroom demos, unit tests	Real ML practice, larger experiments

There's also a third family — make_*

make_classification, **make_regression**, **make_blobs**, **make_moons** generate **synthetic data** on the fly with whatever shape, noise, and class balance you want. They're for testing algorithms in controlled conditions.

```
from sklearn.datasets import make_regression
X, y = make_regression(n_samples=100, n_features=3, noise=10, random_state=42)
X.shape  # (100, 3)
y.shape  # (100,)
```

NOTE: Common PYQ trap: 'The similarity between loaders and fetchers is...' Correct answer: **both return a Bunch object**. They do NOT both load small datasets. They do NOT generate synthetic data — that's **make_***.

7.1 Splitting data into train and test

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,           # 20% goes to test, 80% to train
    random_state=42,        # for reproducibility
    stratify=y              # keeps class proportions balanced (classification only)
)
```

Important parameters

- **test_size**: fraction (0.2) or absolute count (50). Default 0.25.
- **train_size**: if you'd rather specify the train side.
- **random_state**: any integer; locks the shuffle so you get the same split every run. Without it, different splits each call.
- **shuffle**: True by default. Set False for time-series so you don't shuffle past with future.
- **stratify**: when classification has imbalanced classes, pass **stratify=y** to keep class ratios in train and test.

WATCH OUT: PYQ pattern: 'Will the rows in `X_train` be the same in case 1 and case 2 if we use different `random_state` values?' If `random_state` differs, the shuffle differs, so the rows differ. If `random_state` is the same, the rows are guaranteed identical.

8. Handling Missing Data (Imputation)

Real datasets always have missing values (NaN). sklearn won't fit models with NaN — you must remove or fill them first. This is **imputation**.

8.1 Two main strategies

- **Drop rows or columns with missing values** (Complete Case Analysis). Easy but loses information.
- **Fill (impute) the missing values** with something reasonable: mean, median, mode, or a predicted value.

8.2 SimpleImputer — the workhorse

```
from sklearn.impute import SimpleImputer
import numpy as np

# Numeric data: fill missing with the mean
imputer = SimpleImputer(strategy='mean', missing_values=np.nan)
imputer.fit(X_train)
X_train_filled = imputer.transform(X_train)
X_test_filled = imputer.transform(X_test) # use same fitted imputer!
```

Strategies of SimpleImputer

strategy	What it does	Use when
'mean'	Fill NaN with the column mean	Numeric columns, no extreme outliers
'median'	Fill NaN with the column median	Numeric columns, outliers present (more robust)
'most_frequent'	Fill NaN with the most common value (mode)	Categorical columns OR numeric
'constant'	Fill NaN with a fixed value passed via fill_value=	When you want a placeholder like -1 or 'missing'

NOTE: SimpleImputer learns one number per column during fit(). For 'mean' strategy, it stores the mean of each column in `imputer.statistics_`. When you call transform, it uses those stored values to fill NaN.

8.3 KNNImputer — smarter imputation

Instead of using the column mean, KNNImputer fills a missing value by looking at the **k nearest neighbors** (other rows that are most similar to the row with the missing value, based on the columns that ARE filled), and taking their average.

```
from sklearn.impute import KNNImputer

imputer = KNNImputer(n_neighbors=3)
X_filled = imputer.fit_transform(X)
```

TIP: When KNNImputer wins: when columns are correlated (e.g., taller people weigh more, so a missing weight can be filled from neighbors with similar heights). When columns are random and

uncorrelated, plain mean imputation works just as well.

8.4 Indicator columns

Sometimes the fact that a value WAS missing is itself useful information. You can pass **add_indicator=True** to SimpleImputer or use **MissingIndicator** separately to get a boolean column saying "this row had a missing value here."

WATCH OUT: Categorical missing data: for string columns, you cannot use 'mean' or 'median'. Use **strategy='most_frequent'** or **strategy='constant'** with a fill_value like 'missing'.

9. Categorical Encoding

ML models work with numbers. If your data has text categories like 'Mumbai', 'Delhi', 'Pune', you must convert them to numbers first. There are several ways, and choosing the wrong one is a classic mistake.

9.1 OrdinalEncoder — for ordered categories

Use when categories have a natural order: **Low < Medium < High**, or **Cold < Warm < Hot**. The encoder maps them to integers 0, 1, 2, ...

```
from sklearn.preprocessing import OrdinalEncoder

X = [['Low'], ['High'], ['Medium'], ['Low']]
enc = OrdinalEncoder(categories=[['Low', 'Medium', 'High']]) # specify order!
X_encoded = enc.fit_transform(X)
# array([[0.],
#        [2.],
#        [1.],
#        [0.]])
```

WATCH OUT: Don't use **OrdinalEncoder** on **unordered categories** like color or city! The model will assume Mumbai=0 is somehow 'less' than Delhi=1 'less' than Pune=2, which is nonsensical and damages your model.

9.2 OneHotEncoder — for unordered (nominal) categories

Creates one new binary column per unique category. If a row belongs to that category, the column is 1; otherwise 0. There's no false ordering implied.

```
from sklearn.preprocessing import OneHotEncoder

X = [['Mumbai'], ['Delhi'], ['Pune'], ['Mumbai']]
enc = OneHotEncoder(sparse_output=False) # sparse_output=False gives a dense array
X_encoded = enc.fit_transform(X)
# array([[0., 1., 0.], # Mumbai
#        [1., 0., 0.], # Delhi (alphabetical order)
#        [0., 0., 1.], # Pune
#        [0., 1., 0.]]) # Mumbai
```

Important parameters

- **sparse_output** (was **sparse** in older sklearn): True returns a sparse matrix (memory-efficient for many categories), False returns a dense NumPy array.
- **drop**: drop one category to avoid the dummy variable trap (relevant for linear regression). Options: 'first', 'if_binary', None.
- **handle_unknown**: what to do if test data has a category never seen during fit. 'error' (default) crashes; 'ignore' creates an all-zero row.

9.3 LabelEncoder — for the TARGET only

LabelEncoder maps each unique label to an integer. It looks similar to **OrdinalEncoder** but is meant **ONLY** for encoding the target variable y, not features X.

```
from sklearn.preprocessing import LabelEncoder
```

```
y = ['cat', 'dog', 'cat', 'bird', 'dog']
le = LabelEncoder()
y_encoded = le.fit_transform(y)
# array([1, 2, 1, 0, 2]) # alphabetical: bird=0, cat=1, dog=2

le.classes_ # array(['bird', 'cat', 'dog'])
```

NOTE: The big distinction:

- **OneHotEncoder** → use for input features X with no order (cities, colors).
- **OrdinalEncoder** → use for input features X with an order (Low/Med/High).
- **LabelEncoder** → use for the target y in classification.

10. Numeric Transformers & Feature Scaling

Many ML algorithms care about the **SCALE** of features. If one feature is in the range $[0, 1]$ (like a probability) and another is in $[0, 1,000,000]$ (like income), the large-scale feature dominates distance and gradient computations. Feature scaling fixes this by transforming all features to similar ranges.

10.1 Algorithms that **NEED** scaling vs **DON'T** need scaling

Need scaling	Don't need scaling
KNN (uses distance)	Decision Tree
K-Means (uses distance)	Random Forest
SVM (uses distance / margin)	Gradient Boosting (mostly)
PCA (uses variance)	Naive Bayes
Logistic Regression with L1/L2 regularization	Plain Linear Regression (math is invariant, but coefficients become hard to interpret)
Neural Networks (MLPRegressor / MLPClassifier)	
Linear Regression with gradient descent (SGDRegressor)	

NOTE: Memorize this table. Direct PYQ question: 'Which algorithms don't get impacted by feature scaling?' Answer: tree-based methods (DecisionTree, RandomForest) and Naive Bayes.

10.2 StandardScaler — Z-score normalization

Transforms each feature to have **mean = 0** and **standard deviation = 1**. Formula: $z = (x - \text{mean}) / \text{std}$.

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()  
X_scaled = scaler.fit_transform(X_train)
```

```
scaler.mean_ # the mean of each column (learned during fit)  
scaler.scale_ # the std of each column
```

- **Pros:** Most common choice. Works well with normally-distributed data.
- **Cons:** Sensitive to outliers — a single huge value can shift the mean and inflate std.
- **Output range:** NOT bounded. Most values land in roughly $[-3, +3]$ for normal data, but extremes can go further.

10.3 MinMaxScaler — fixed range

Transforms each feature into a fixed range, by default $[0, 1]$. Formula: $x_{\text{scaled}} = (x - \text{min}) / (\text{max} - \text{min})$.

```
from sklearn.preprocessing import MinMaxScaler
```

```
scaler = MinMaxScaler(feature_range=(0, 1)) # default  
X_scaled = scaler.fit_transform(X_train)
```

- **Pros:** Output is bounded, exactly between 0 and 1 (good for neural nets, image pixels).
- **Cons:** Very sensitive to outliers — one extreme value sets the max and squashes everything else into a tiny range.

10.4 RobustScaler — outlier-resistant

Uses the median and the interquartile range ($IQR = Q3 - Q1$) instead of mean and std. Outliers don't affect the median or IQR much, so this scaler is robust to them.

```
from sklearn.preprocessing import RobustScaler

scaler = RobustScaler()
X_scaled = scaler.fit_transform(X_train)
```

10.5 MaxAbsScaler — preserves sparsity

Divides each feature by its maximum absolute value, mapping to $[-1, 1]$. Doesn't shift or center the data, so it preserves zeros — useful for sparse data.

Which scaler to use?

Scaler	Output range	Outlier-robust ?	Best for
StandardScaler	Roughly $[-3, +3]$, unbounded	No	General use, default choice
MinMaxScaler	$[0, 1]$ exactly	No	Neural nets, image data
RobustScaler	Centered around 0, varies	Yes	Data with outliers
MaxAbsScaler	$[-1, 1]$	No	Sparse data, preserves zeros

WATCH OUT: Scaler PYQ trap: 'After applying StandardScaler, what is the mean?' Answer: 0 (approximately, due to floating point). Std: 1. **After MinMaxScaler:** min is 0, max is 1.

11. Outlier Detection & Removal

An **outlier** is a data point that's far from most others. Outliers can wreck a model (especially linear regression and KNN). You have two responses: remove them or use methods robust to them.

11.1 The Z-score method

Z-score tells you how many standard deviations a point is away from the mean. $z = (x - \text{mean}) / \text{std}$. A common rule: any point with $|z| > 3$ is an outlier.

```
import numpy as np

mean = X.mean()
std = X.std()
z = (X - mean) / std
mask = np.abs(z) < 3      # keep only points within 3 std
X_clean = X[mask]
```

- **Best when:** data is roughly normally distributed (bell-shaped).
- **Weakness:** the mean and std themselves are pulled by outliers, so extreme outliers can mask each other.

11.2 The IQR method

Compute **Q1** (25th percentile) and **Q3** (75th percentile). The interquartile range is $\text{IQR} = \text{Q3} - \text{Q1}$. Any point below $\text{Q1} - 1.5 \times \text{IQR}$ or above $\text{Q3} + 1.5 \times \text{IQR}$ is an outlier. This is the same rule used to draw whiskers in a box plot.

```
Q1 = np.percentile(X, 25)
Q3 = np.percentile(X, 75)
IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

X_clean = X[(X >= lower) & (X <= upper)]
```

- **Best when:** data is skewed or non-normal.
- **Strength:** percentiles are robust — they don't get pulled by outliers.

11.3 The percentile (capping/trimming) method

Decide a percentile threshold (e.g., 1st and 99th) and either drop or cap values outside it. **Capping** (also called Winsorization) replaces extremes with the threshold value instead of removing the row.

```
lower = np.percentile(X, 1)
upper = np.percentile(X, 99)

# Trim (drop) approach:
X_clean = X[(X >= lower) & (X <= upper)]

# Cap approach (Winsorization):
X_capped = np.clip(X, lower, upper)
```

NOTE: Why care about outliers in feature scaling? StandardScaler and MinMaxScaler are both outlier-sensitive. If you scale a column with one extreme outlier, everything else gets squashed.

Either remove the outlier first, or use RobustScaler instead.

11.4 Quick decision guide

Situation	Method
Data is roughly normal	Z-score (3 std rule)
Data is skewed	IQR ($1.5 \times$ IQR rule)
You don't trust your distribution assumptions	Percentile (e.g., 1st/99th)
You want to keep all rows but tame extremes	Capping (Winsorization) instead of dropping
You don't want to deal with outliers at all	Use RobustScaler + tree-based models

Part 1 Recap — What You Now Know

You should now be comfortable with:

- Reading Python, NumPy, and Pandas code without panic.
- Distinguishing Series from DataFrame; using loc, iloc, boolean masks.
- The sklearn estimator API (fit, transform, predict, fit_transform).
- Bunch object structure (data, target, feature_names, target_names).
- Difference between **load_***, **fetch_***, and **make_*** functions.
- train_test_split parameters: test_size, random_state, stratify.
- Imputation: SimpleImputer (mean, median, most_frequent, constant), KNNImputer.
- Encoding: OrdinalEncoder (ordered features), OneHotEncoder (nominal features), LabelEncoder (target).
- Scaling: StandardScaler (Z-score), MinMaxScaler (0-1), RobustScaler (outlier-proof), MaxAbsScaler (sparse data).
- Outlier methods: Z-score, IQR, percentile, capping.

Quick self-check questions

Try answering these without scrolling back. Then verify with what you just learned.

1. What does `load_iris()` return, and what's the type of `iris.data`?
2. You scale your `X_train` with `fit_transform`. What method should you call on `X_test`?
3. Which scaler should you use if your data has many extreme outliers?
4. You have a column with categories ['Low', 'Medium', 'High']. Which encoder fits?
5. `df.iloc[0:3]` returns how many rows?
6. `df.loc[0:3]` returns how many rows?
7. What strategy parameter of `SimpleImputer` should you use for a string column?
8. Name three algorithms that DON'T need feature scaling.

Answers

1. A **Bunch** object; `iris.data` is a NumPy array of shape (150, 4).
2. **Only transform()**, never `fit_transform` — to avoid data leakage.
3. **RobustScaler** (uses median and IQR, both robust to outliers).
4. **OrdinalEncoder** with explicit category order, since Low < Medium < High.
5. **3 rows** (positions 0, 1, 2 — end exclusive).
6. **4 rows** (labels 0, 1, 2, 3 — end inclusive!).
7. **'most_frequent'** or **'constant'** with a `fill_value`.
8. **DecisionTree, RandomForest, Naive Bayes** (also: gradient boosting and other tree-based).

What's Next — Part 2

Part 2 will cover: **Feature Selection** (filter / wrapper / embedded methods, `VarianceThreshold`, `SelectKBest`, `RFE`), **PCA** (the math, the parameters, the output), **Pipelines** and **ColumnTransformer** (wiring up preprocessing into a single object), and the heart of regression: **Linear Regression**, **Polynomial Regression**, **Ridge / Lasso / ElasticNet** (regularization), and **SGDRegressor**.

Once you're done reading Part 1, do a quick PYQ practice: open the 2024 Apr 28 paper, skim the first 10 questions. You'll find that several of them are now answerable based on what you've learned in this part alone.

MLP Exam Prep

Concept Mastery Guide — Part 2 of 4

Feature Selection, PCA, Pipelines & Regression

Part 2 picks up right where Part 1 left off. By now you can read pandas/sklearn code, split data, impute, encode, and scale. Now we'll trim down the feature set, project it into fewer dimensions, wire all preprocessing into a single Pipeline object, and build your first real models — the entire regression family.

What's in Part 2

#	Section	Why it matters
12	Feature selection — filter methods	VarianceThreshold, SelectKBest
13	Feature selection — wrapper & embedded	RFE, SelectFromModel
14	PCA — dimensionality reduction	L2.10, asked every paper
15	Pipelines & make_pipeline	L3.1, asked every paper
16	ColumnTransformer & FeatureUnion	L2.9, L3.3
17	Linear Regression — the math & the API	L4.1, L5.2
18	Regression metrics	L4.2 — heavy weightage
19	Polynomial Regression	L4.3
20	Regularization: Ridge, Lasso, ElasticNet	L4.4 — heavy weightage
21	Gradient Descent & SGDRegressor	L5.5
22	Hyperparameter tuning preview	L4.5 (full coverage in Part 4)

12. Feature Selection — Filter Methods

When your dataset has 50, 100, or 500 columns, many of them are useless or redundant. Feature selection picks the most useful subset. There are three families:

- **Filter methods:** use simple statistics, no model involved. Fast and cheap.
- **Wrapper methods:** wrap a model and try different feature subsets. Slow but accurate.
- **Embedded methods:** the model itself does selection during training (like Lasso).

12.1 VarianceThreshold — drop near-constant columns

If a column has the same value in 99% of rows, it carries almost no information. VarianceThreshold drops columns whose variance is below a threshold.

```
from sklearn.feature_selection import VarianceThreshold

selector = VarianceThreshold(threshold=0.1)  # drop cols with var < 0.1
X_selected = selector.fit_transform(X)

selector.get_support()    # boolean array: True = kept, False = dropped
selector.variances_       # the variance of each original column
```

NOTE: threshold=0 (the default) drops only columns where every row has the same value (zero variance). Higher threshold = more aggressive dropping.

12.2 SelectKBest — keep the top k columns

SelectKBest uses a **scoring function** to rank each feature against the target, then keeps the top k. The scoring function depends on whether you're doing classification or regression, and on whether features are numeric or categorical.

```
from sklearn.feature_selection import SelectKBest, f_classif, f_regression, chi2, mutual_info_classif

# For classification with numeric features:
selector = SelectKBest(score_func=f_classif, k=10)

# For regression with numeric features:
selector = SelectKBest(score_func=f_regression, k=10)

# For classification with non-negative features (counts, frequencies):
selector = SelectKBest(score_func=chi2, k=10)

X_new = selector.fit_transform(X, y)
selector.scores_      # the score of each feature
selector.get_support() # which features were kept
```

Which scoring function for which case?

Score function	Use for	What it measures
f_classif	Classification, numeric features	ANOVA F-value (does the mean differ across classes?)
f_regression	Regression, numeric features	F-test from linear correlation with target
chi2	Classification, NON-NEGATIVE features (counts, frequencies)	Independence test between feature and target
mutual_info_classif	Classification, any features	Mutual information (captures non-linear relationships)
mutual_info_regression	Regression, any features	Same idea, regression target

WATCH OUT: chi2 trap: chi2 requires features to be NON-NEGATIVE. If you've standardized your data (StandardScaler produces negative values), chi2 will throw an error. Use MinMaxScaler or skip scaling before chi2.

12.3 SelectPercentile — keep the top X% of features

Same idea as SelectKBest, but you specify a percentage instead of a count.

```
from sklearn.feature_selection import SelectPercentile, f_regression
selector = SelectPercentile(score_func=f_regression, percentile=20) # keep top 20%
X_new = selector.fit_transform(X, y)
```

13. Feature Selection — Wrapper & Embedded Methods

13.1 RFE — Recursive Feature Elimination (wrapper)

RFE is a wrapper method that uses a model's own feature importances. The procedure:

- 1. Train the model on all features.
- 2. Find the LEAST important feature (smallest coefficient or smallest feature_importance).
- 3. Drop it. Re-train on the remaining features.
- 4. Repeat until only n_features_to_select features remain.

```
from sklearn.feature_selection import RFE
from sklearn.linear_model import LogisticRegression

model = LogisticRegression()
selector = RFE(estimator=model, n_features_to_select=5, step=1)
selector.fit(X, y)

selector.support_      # boolean mask of selected features
selector.ranking_      # rank 1 = best, higher = eliminated earlier
selector.n_features_   # how many features were kept
```

NOTE: **step** controls how many features to remove per iteration. **step=1** removes one at a time (slow but accurate); **step=5** removes five at a time (faster but coarser).

13.2 RFECV — RFE with cross-validation

Same as RFE, but uses cross-validation to **automatically choose the optimal number of features** instead of requiring you to specify it.

```
from sklearn.feature_selection import RFECV
selector = RFECV(estimator=model, step=1, cv=5, scoring='accuracy')
selector.fit(X, y)

selector.n_features_   # the chosen number of features
```

13.3 SelectFromModel (embedded)

Uses any model with a **coef_** or **feature_importances_** attribute and keeps only features whose importance is above a threshold. Very common with Lasso (where many coefficients become exactly zero) and tree-based models.

```
from sklearn.feature_selection import SelectFromModel
from sklearn.linear_model import Lasso

model = Lasso(alpha=0.1)
selector = SelectFromModel(estimator=model, threshold='median')
X_new = selector.fit_transform(X, y)
```

Threshold options

- **Numeric value:** e.g. **threshold=0.05** keeps features with importance > 0.05.
- **'mean':** keep features with importance above the mean importance.
- **'median':** keep features above median importance.

- **'1.25*mean'**: scale factor strings are allowed.

13.4 Decision guide — which method to use

Situation	Method
Quick first pass on a huge dataset	VarianceThreshold + SelectKBest
Classification with discrete/count features	chi2 + SelectKBest
Regression with numeric features	f_regression + SelectKBest
You want the model itself to pick features	Lasso + SelectFromModel (embedded)
You want best subset, can afford slow training	RFE or RFECV
You don't know how many features to keep	RFECV (auto-decides via CV)

TIP: Filter vs Wrapper vs Embedded summary: Filter = score each feature individually, ignore the model. Wrapper = train the model many times with different subsets. Embedded = the model penalizes useless features during its own training (Lasso, decision tree). Speed: filter > embedded > wrapper. Accuracy: wrapper > embedded > filter.

14. PCA — Principal Component Analysis

PCA is the most famous **dimensionality reduction** technique. The goal: take your **n**-dimensional data and project it into **k < n** dimensions, while losing as little information as possible. "Information" here means **variance**.

14.1 The intuition (no math)

Imagine a cloud of 2-D points that roughly forms a diagonal line. The original axes are x and y. But really, the data lives along a single direction (the diagonal). PCA finds that diagonal — calls it **Principal Component 1 (PC1)** — and a perpendicular direction with whatever variance is left, called **PC2**. If you keep only PC1, you've gone from 2-D to 1-D while keeping most of the variance.

In general, PCA produces **n new axes** (called principal components) ranked by how much variance they capture. PC1 has the most variance, PC2 the second most, and so on. You keep the first **k** components; the rest are discarded.

14.2 PCA in scikit-learn

```
from sklearn.decomposition import PCA
```

```
pca = PCA(n_components=2)
X_reduced = pca.fit_transform(X)
```

```
pca.components_          # the principal directions (each row is a component)
pca.explained_variance_   # how much variance each component captures (raw)
pca.explained_variance_ratio_ # same, as a fraction of total variance
pca.singular_values_     # the singular values from SVD
```

Important parameters

- **n_components=k** (integer): keep top k components.
- **n_components=0.95** (float between 0 and 1): keep enough components to capture 95% of total variance — sklearn picks k automatically.
- **n_components='mle'**: use Minka's MLE to auto-pick k.
- **whiten=True**: after projection, divide by std so all components have unit variance. Useful for KMeans.

14.3 explained_variance_ratio_ — the most-asked output

If you fit PCA on data with 4 features and ask for n_components=2, you'll get something like:

```
pca.explained_variance_ratio_
# array([0.72, 0.23])
# Means: PC1 captures 72% of variance, PC2 captures 23%.
# Together they capture 95% of the original variance.

pca.explained_variance_ratio_.sum()    # 0.95
```

WATCH OUT: Always scale before PCA. PCA is variance-based: a feature measured in millions (income) will dominate one measured in 0-1 (probability). Apply StandardScaler first, always.

14.4 What does PCA NOT do

- **It doesn't preserve interpretability.** PC1 is some weird linear combination of all original features. You can't easily say what it means.
- **It doesn't help with non-linear structure.** If your data lives on a curve, PCA (which is linear) will struggle. Use t-SNE or UMAP for non-linear cases (but those aren't typically in this exam).
- **It doesn't choose features.** PCA creates new ones from combinations. If you want to keep original feature names, use feature selection methods instead.

14.5 PCA vs feature selection

	PCA	Feature Selection (e.g., SelectKBest)
What it produces	New synthetic features (linear combinations)	Subset of original features
Interpretability	Hard (PC1 = $0.4 \times \text{age} + 0.3 \times \text{income} + \dots$)	Easy (just original column names)
Uses target y?	No (unsupervised)	Yes (most methods)
Best for	Heavy compression, visualization in 2-D/3-D	Keeping interpretable feature names

NOTE: Common PYQ: 'What is the explained variance score for the following code?' If a snippet does `PCA(n_components=2).fit(X).explained_variance_ratio_` on data where the first feature has all the variance, you'll see `[1., 0.]` — PC1 captures 100%, PC2 captures 0%. The output is always a length-k array that sums to ≤ 1 .

15. Pipelines & make_pipeline

Your preprocessing easily becomes 5+ steps: impute, encode, scale, select features, reduce dimensions, then fit the model. Doing this manually is error-prone — you'll forget to apply scaling on test data, or you'll fit_transform on test by accident. **Pipeline** fixes this by chaining everything into one object.

15.1 The Pipeline class

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='mean')),
    ('scaler', StandardScaler()),
    ('model', LogisticRegression())
])

pipe.fit(X_train, y_train)      # fits all steps in sequence
pipe.predict(X_test)           # transforms test, then predicts
pipe.score(X_test, y_test)     # transforms test, predicts, scores
```

Each step is a tuple (**name**, **estimator**). The names are arbitrary but must be unique. The last step is usually a model (predictor); all earlier steps must be transformers.

How fit and predict flow through a Pipeline

When you call **pipe.fit(X_train, y_train)**:

- Step 1: **imputer.fit_transform(X_train)** — output goes to Step 2.
- Step 2: **scaler.fit_transform(imputed_X)** — output goes to Step 3.
- Step 3: **model.fit(scaled_X, y_train)** — done.

When you call **pipe.predict(X_test)**:

- Step 1: **imputer.transform(X_test)** (note: transform only, no fit!)
- Step 2: **scaler.transform(imputed_X)**
- Step 3: **model.predict(scaled_X)**

TIP: This is why Pipelines prevent data leakage. They guarantee transform-only on test data because there is no fit method called on the test set. You literally cannot make the leakage mistake.

15.2 make_pipeline — the shortcut

If you don't care about naming the steps, **make_pipeline** auto-generates names from the class names (in lowercase).

```
from sklearn.pipeline import make_pipeline

pipe = make_pipeline(
    SimpleImputer(strategy='mean'),
    StandardScaler(),
    LogisticRegression()
)
```



```
# Equivalent to:  
# Pipeline([('simpleimputer', ...), ('standardscaler', ...), ('logisticregression', ...)])
```

15.3 Accessing pipeline components

```
pipe.named_steps['scaler']      # the scaler object  
pipe.named_steps['scaler'].mean_ # learned mean  
  
pipe['scaler']                  # same thing, shorter syntax (newer sklearn)  
pipe.steps                     # list of (name, estimator) tuples  
  
# Set hyperparameters with double underscore syntax (used in GridSearchCV):  
pipe.set_params(model__C=10)    # sets C parameter of the 'model' step
```

NOTE: The **step__param** syntax (e.g., 'model__C') is critical for hyperparameter tuning a Pipeline. We'll see it again in GridSearchCV (Part 4).

16. ColumnTransformer & FeatureUnion

Real datasets have heterogeneous columns: numeric (age, income), categorical (city, gender), and maybe text. You need DIFFERENT preprocessing per column type. A Pipeline applies the same step to all columns — that's not what you want here. Enter **ColumnTransformer**.

16.1 ColumnTransformer — different preprocessing per column

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline

numeric_cols = ['age', 'income']
categorical_cols = ['city', 'gender']

numeric_pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='mean')),
    ('scaler', StandardScaler())
])

categorical_pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OneHotEncoder(handle_unknown='ignore'))
])

preprocessor = ColumnTransformer([
    ('num', numeric_pipe, numeric_cols),
    ('cat', categorical_pipe, categorical_cols)
])

X_processed = preprocessor.fit_transform(X)
```

Each tuple is **(name, transformer, columns)**. The transformer is applied only to the listed columns; other columns are dropped by default.

The remainder parameter

Sets what happens to columns NOT listed in any transformer:

- **remainder='drop'** (default): unlisted columns are removed.
- **remainder='passthrough'**: unlisted columns are kept as-is, untouched.
- **remainder=some_transformer**: apply that transformer to the rest.

```
ColumnTransformer([
    ('num', StandardScaler(), ['age', 'income'])
], remainder='passthrough') # other columns kept unchanged
```

16.2 make_column_transformer — the shortcut

```
from sklearn.compose import make_column_transformer

preprocessor = make_column_transformer(
    (StandardScaler(), ['age', 'income']),
    (OneHotEncoder(), ['city', 'gender']),
    remainder='passthrough'
)
```

16.3 FeatureUnion — combine output of multiple transformers

FeatureUnion applies several transformers to the SAME data and concatenates their outputs side-by-side. Different from ColumnTransformer (different columns) — FeatureUnion is same columns, multiple views.

```
from sklearn.pipeline import FeatureUnion
from sklearn.decomposition import PCA
from sklearn.feature_selection import SelectKBest

union = FeatureUnion([
    ('pca', PCA(n_components=3)),
    ('select', SelectKBest(k=5))
])

# After fit_transform, output has 3 + 5 = 8 columns:
# 3 PCA components + 5 selected features
X_combined = union.fit_transform(X, y)
```

NOTE: Pipeline vs ColumnTransformer vs FeatureUnion — the most-asked PYQ comparison:

- **Pipeline** = sequential. Output of step 1 → input of step 2 → ... → final model.
- **ColumnTransformer** = parallel on DIFFERENT columns. Each transformer touches its own subset.
- **FeatureUnion** = parallel on the SAME data. All transformers see all columns; outputs concatenate.

16.4 The end-to-end pattern

In a real ML script, you'll combine ColumnTransformer (for heterogeneous preprocessing) with Pipeline (to attach the model at the end):

```
full_pipe = Pipeline([
    ('preprocessor', preprocessor), # the ColumnTransformer
    ('model', LogisticRegression())
])

full_pipe.fit(X_train, y_train)
full_pipe.score(X_test, y_test)
```

17. Linear Regression

The simplest, most-asked regression model. The idea: predict a numeric target y as a **weighted sum** of the features, plus a bias term.

17.1 The model

$$y = w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_n \cdot x_n + b$$

Here $w_1, \dots, w_n$ are the **coefficients** (one per feature), and b is the **intercept** (bias). Linear regression learns these by minimizing the squared differences between predicted and actual values — this is the **Ordinary Least Squares (OLS)** loss:

$$\text{Loss} = \sum (y_{\text{actual}} - y_{\text{predicted}})^2$$

17.2 LinearRegression in scikit-learn

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)

model.coef_          # array of coefficients [w1, w2, ..., wn]
model.intercept_     # scalar b

y_pred = model.predict(X_test)
model.score(X_test, y_test)  # R squared by default
```

Important parameters

- **fit_intercept=True** (default): learn the intercept b . Set **False** only if your data is already centered.
- **n_jobs=-1**: parallelize across cores.
- **positive=True** (newer sklearn): force coefficients to be non-negative.

NOTE: **LinearRegression** uses a **closed-form solution** (the normal equation: $w = (X^T X)^{-1} X^T y$). It does NOT use gradient descent. For large datasets where matrix inversion is too slow, use **SGDRegressor** instead.

17.3 Assumptions of linear regression

- **Linearity:** the relationship between X and y is approximately linear.
- **Independence:** observations are independent (no time-series autocorrelation).
- **Homoscedasticity:** residuals have constant variance across X .
- **Normality of residuals:** errors are roughly normally distributed (matters more for inference than prediction).
- **No multicollinearity:** features aren't perfectly correlated with each other.

17.4 Multicollinearity — the silent killer

If two features are highly correlated (e.g., 'height in cm' and 'height in inches'), linear regression's coefficients become unstable. The math **doesn't fail**, but tiny data changes cause coefficients to swing wildly. Solutions: drop one redundant feature, use Ridge regression, or use PCA before regression.

WATCH OUT: Outlier sensitivity: Plain linear regression with squared loss is heavily affected by outliers (squaring amplifies large errors). For outlier-resistant linear models, look into **HuberRegressor** or **RANSACRegressor**.

18. Regression Metrics

How do you score a regression model? Several metrics, each measuring something slightly different. The exam asks you to compute these by hand or read sklearn outputs.

18.1 Mean Absolute Error (MAE)

$$\text{MAE} = (1/n) \cdot \sum |y_{\text{actual}} - y_{\text{predicted}}|$$

The average size of error, in the same units as y. Easy to interpret. Less sensitive to outliers than squared metrics.

```
from sklearn.metrics import mean_absolute_error
mae = mean_absolute_error(y_true, y_pred)
```

18.2 Mean Squared Error (MSE)

$$\text{MSE} = (1/n) \cdot \sum (y_{\text{actual}} - y_{\text{predicted}})^2$$

Average squared error. Penalizes large errors heavily (because of the square). The loss function that LinearRegression actually minimizes.

```
from sklearn.metrics import mean_squared_error
mse = mean_squared_error(y_true, y_pred)
```

18.3 Root Mean Squared Error (RMSE)

$$\text{RMSE} = \sqrt{\text{MSE}}$$

Just the square root of MSE, returning to the same units as y. Easier to compare to MAE because units match.

```
# Older sklearn:
from sklearn.metrics import mean_squared_error
rmse = mean_squared_error(y_true, y_pred, squared=False)

# Newer sklearn (1.4+):
from sklearn.metrics import root_mean_squared_error
rmse = root_mean_squared_error(y_true, y_pred)
```

18.4 R^2 Score (Coefficient of Determination)

$$R^2 = 1 - (\text{SS}_{\text{residuals}} / \text{SS}_{\text{total}})$$

where $\text{SS}_{\text{residuals}} = \sum (y_{\text{actual}} - y_{\text{predicted}})^2$ (your model's squared errors), and $\text{SS}_{\text{total}} = \sum (y_{\text{actual}} - y_{\text{mean}})^2$ (total variance of the target).

Interpretation:

- $R^2 = 1.0$: perfect predictions.
- $R^2 = 0.0$: model is no better than predicting the mean of y for every input.
- $R^2 < 0$: model is WORSE than predicting the mean. Yes, R^2 can be negative.

```
from sklearn.metrics import r2_score
r2 = r2_score(y_true, y_pred)

# LinearRegression.score() returns R squared by default:
```

```
model.score(X_test, y_test)
```

18.5 Adjusted R^2

Plain R^2 has a flaw: adding more features almost always increases R^2 , even if those features are useless (because the model can fit noise). **Adjusted R^2** penalizes extra features:

$$\text{Adj } R^2 = 1 - (1 - R^2) \cdot (n - 1) / (n - p - 1)$$

where n = number of samples, p = number of features. Adjusted R^2 only goes up when a new feature actually helps. There is no built-in adjusted R^2 in sklearn — compute it manually.

18.6 Choosing a metric

Metric	Sensitive to outliers?	Best when
MAE	Less sensitive	You want a robust, interpretable error
MSE / RMSE	Very sensitive (squared)	Large errors are particularly costly
R^2 / Adjusted R^2	Sensitive (uses MSE)	You want a unitless quality score
MAPE (Mean Absolute Percent Error)	Sensitive when y is near 0	Errors should be measured as percentages

NOTE: PYQ pattern: 'Compute the R^2 score for this prediction.' Set up $\text{SS}_{\text{res}} = \sum((y - y_{\text{pred}})^2)$, $\text{SS}_{\text{tot}} = \sum((y - \text{mean}(y))^2)$, then $R^2 = 1 - \text{SS}_{\text{res}}/\text{SS}_{\text{tot}}$. Practice this on a tiny example with 4 numbers.

19. Polynomial Regression

Linear regression fits a straight line. If your true relationship is curved, a straight line won't fit well. **Polynomial regression** is a clever trick: instead of changing the model, we add new features that are powers of the original.

19.1 The trick

Suppose you have a single feature x . If you create new features x^2 and x^3 , you now have a 3-feature dataset. Apply plain LinearRegression to it:

$$y = w_1 \cdot x + w_2 \cdot x^2 + w_3 \cdot x^3 + b$$

This is a cubic curve in x — but mathematically it's still linear in the WEIGHTS, so LinearRegression handles it just fine.

19.2 PolynomialFeatures in sklearn

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import make_pipeline

# Take 1-D x -> produce features [1, x, x^2, x^3]
poly = PolynomialFeatures(degree=3, include_bias=True)

model = make_pipeline(
    PolynomialFeatures(degree=3),
    LinearRegression()
)
model.fit(X_train, y_train)
```

Important parameters

- **degree**: the highest power. degree=2 gives $[1, x, x^2]$ for one feature.
- **include_bias=True** (default): include the constant 1 column. Set False if your model already adds an intercept (LinearRegression does, so you can set False to avoid duplication — both work in practice).
- **interaction_only=True**: include cross-terms like $x_1 \cdot x_2$ but skip pure powers like x_1^2 .

19.3 The combinatorial explosion

With n features and degree d , the number of polynomial features grows fast:

Original features (n)	Degree 2 features	Degree 3 features
2 features (x_1, x_2)	$1 + x_1 + x_2 + x_1^2 + x_1x_2 + x_2^2 = 6$	10
5	21	56
10	66	286
50	1,326	23,426

WATCH OUT: Why this matters: high-degree polynomials on many features cause heavy overfitting AND blow up your memory. In practice, degree=2 or 3 with regularization (Ridge or

Lasso instead of plain LinearRegression) is the safe choice.

20. Regularization: Ridge, Lasso, ElasticNet

Plain linear regression's only goal is minimizing prediction error on the training set. If features are correlated or there are many of them, it'll learn huge wiggly coefficients that overfit. **Regularization** adds a **penalty on the size of coefficients** to the loss function, forcing them to stay small.

20.1 Ridge regression — L2 penalty

$$\text{Loss} = \sum (y - y_{\text{pred}})^2 + \alpha \cdot \sum w_i^2$$

The extra term $\alpha \cdot \sum w_i^2$ is the L2 (squared) penalty. α (alpha) controls how strong the penalty is. Larger $\alpha \rightarrow$ smaller coefficients $\rightarrow$ simpler model.

```
from sklearn.linear_model import Ridge
```

```
model = Ridge(alpha=1.0)
model.fit(X_train, y_train)
model.coef_      # coefficients (smaller in magnitude than plain LinearRegression)
```

- $\alpha = 0$: equivalent to plain LinearRegression.
- $\alpha \rightarrow \infty$: all coefficients shrink toward (but not exactly) zero.
- **Default $\alpha = 1.0$.**

NOTE: Ridge shrinks coefficients but never sets them exactly to zero. So Ridge is good for handling correlated features and reducing overfitting, but it doesn't do feature selection.

20.2 Lasso regression — L1 penalty

$$\text{Loss} = \sum (y - y_{\text{pred}})^2 + \alpha \cdot \sum |w_i|$$

L1 uses absolute values instead of squares. The geometry of the absolute-value penalty causes some coefficients to become **exactly zero**. This means Lasso performs **automatic feature selection**.

```
from sklearn.linear_model import Lasso
```

```
model = Lasso(alpha=0.1)
model.fit(X_train, y_train)
model.coef_  # Many of these will be exactly 0
```

TIP: Lasso is the embedded feature selection method we mentioned earlier. Combine it with SelectFromModel to keep only the features Lasso considered important.

20.3 ElasticNet — combine L1 + L2

ElasticNet mixes both penalties. The mixing ratio is controlled by **l1_ratio**:

- **l1_ratio = 0**: pure Ridge.
- **l1_ratio = 1**: pure Lasso.
- **l1_ratio = 0.5**: equal mix (default).

```
from sklearn.linear_model import ElasticNet
model = ElasticNet(alpha=0.1, l1_ratio=0.5)
```

20.4 Ridge vs Lasso vs ElasticNet — when to use which

	Ridge (L2)	Lasso (L1)	ElasticNet
Penalty	$\sum w_i^2$	$\sum w_i $	Mix of both
Coefficients can be exactly 0?	No	Yes	Yes (from L1 part)
Does feature selection?	No	Yes	Yes
Handles correlated features?	Excellent	Picks one, ignores rest	Good middle ground
Best when	All features matter, just want shrinkage	Many useless features, want sparsity	Many features, some correlated

20.5 Cross-validated versions: RidgeCV, LassoCV, ElasticNetCV

These auto-pick the best **alpha** using cross-validation:

```
from sklearn.linear_model import RidgeCV, LassoCV, ElasticNetCV

model = RidgeCV(alphas=[0.01, 0.1, 1.0, 10.0, 100.0], cv=5)
model.fit(X_train, y_train)
model.alpha_ # the alpha chosen by CV
```

WATCH OUT: Always scale features before regularized regression. Regularization penalizes coefficient magnitudes — but a feature on a large scale (e.g., income in dollars) will naturally have a tiny coefficient even if it's important. After scaling, all features are on equal footing for the penalty.

21. Gradient Descent & SGDRegressor

LinearRegression solves OLS using the normal equation, which requires inverting an $n \times n$ matrix. For huge datasets (millions of rows) this is too expensive. **Gradient descent** finds the same answer iteratively, one small step at a time, without ever inverting a matrix.

21.1 Three flavors of gradient descent

Type	How it computes the gradient	Speed	Memory
Batch GD	Uses ALL training data per step	Slow per step, accurate	High (whole dataset in memory)
Stochastic GD (SGD)	Uses ONE random sample per step	Fast per step, noisy	Tiny
Mini-batch GD	Uses a small batch (e.g., 32) per step	Best of both worlds	Moderate

21.2 SGDRegressor

Sklearn's **SGDRegressor** implements stochastic gradient descent for linear models. It can fit linear regression, Ridge, Lasso, ElasticNet, and Huber — all controlled by the **loss** and **penalty** parameters.

```
from sklearn.linear_model import SGDRegressor

model = SGDRegressor(
    loss='squared_error',      # 'squared_error' = OLS, 'huber' = robust
    penalty='l2',              # 'l2' = Ridge, 'l1' = Lasso, 'elasticnet' = both
    alpha=0.0001,             # regularization strength
    max_iter=1000,             # max passes over the data (epochs)
    tol=1e-3,                  # stopping criterion
    learning_rate='invscaling',
    eta0=0.01,                 # initial learning rate
    random_state=42,
    shuffle=True,              # shuffle training data each epoch (default True)
    warm_start=False
)
model.fit(X_train, y_train)
```

Critical parameters

- **loss**: what to minimize. 'squared_error' for regression, 'huber' for outlier-robust regression, 'epsilon_insensitive' for SVR-style.
- **penalty**: regularization. 'l2', 'l1', 'elasticnet', or None.
- **alpha**: regularization strength (same role as alpha in Ridge/Lasso).
- **max_iter**: maximum number of passes through the entire training set. Each pass is called an **epoch**.
- **tol**: stop early if loss stops improving by this much.
- **learning_rate**: 'constant', 'optimal', 'invscaling' (default), or 'adaptive'. Controls how the step size changes over time.
- **eta0**: the initial step size when learning_rate is 'constant' or 'invscaling'.

- **shuffle=True**: shuffle the training data before each epoch (default and recommended).

21.3 partial_fit and warm_start — incremental learning

Two ways to continue training without losing what you've learned:

- **partial_fit(X, y)**: trains for one epoch using the data you give it. Call repeatedly to learn from streaming data.
- **warm_start=True**: when set, calling fit() again continues from the existing weights instead of resetting. Without warm_start, every fit() restarts from scratch.

```
# warm_start example (asked in Dec 2025 PYQ)
model = SGDRegressor(warm_start=True, max_iter=100)
model.fit(X_batch1, y_batch1)
model.fit(X_batch2, y_batch2)    # CONTINUES from previous weights
```

NOTE: PYQ asked exactly this: 'How do I make fit() continue from previous weights?' Answer: **warm_start=True**. Distractors like **retain_parameters=True** are made up and don't exist.

21.4 Why feature scaling is mandatory for SGD

Gradient descent takes steps proportional to feature magnitudes. If one feature is in [0, 1] and another in [0, 1,000,000], the gradient is dominated by the large-scale feature, the loss surface becomes elongated, and convergence is glacial or fails entirely. **Always StandardScaler before SGDRegressor.**

WATCH OUT: Common SGDRegressor exam trap: SGD uses RANDOM samples each step. Even with the same data, two runs may produce different coefficients unless you set **random_state**. Reproducibility requires a fixed random_state.

22. Hyperparameter Tuning — A Brief Preview

Every model has knobs you set before training (called **hyperparameters**): `alpha` in Ridge, `n_neighbors` in KNN, `max_depth` in DecisionTree, `k` in PCA, etc. Choosing the right values is critical. Two main approaches:

22.1 GridSearchCV — exhaustive search

Try every combination of hyperparameter values you provide:

```
from sklearn.model_selection import GridSearchCV
from sklearn.linear_model import Ridge

param_grid = {
    'alpha': [0.01, 0.1, 1.0, 10.0],
    'fit_intercept': [True, False]
}

grid = GridSearchCV(Ridge(), param_grid, cv=5, scoring='neg_mean_squared_error')
grid.fit(X_train, y_train)

grid.best_params_      # {'alpha': 1.0, 'fit_intercept': True}
grid.best_score_       # CV score of the best combination
grid.best_estimator_   # the actual Ridge() model with best params
```

NOTE: Counting combinations: 4 alphas × 2 fit_intercept = 8 combinations. With 5-fold CV that's 40 model fits. **This counting is asked directly in PYQs.**

22.2 RandomizedSearchCV — random sampling

If your grid is too large, randomly sample `n_iter` combinations instead of trying everything:

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import uniform

param_dist = {
    'alpha': uniform(0.001, 10),    # uniform random between 0.001 and 10
    'fit_intercept': [True, False]
}

search = RandomizedSearchCV(Ridge(), param_dist, n_iter=20, cv=5)
search.fit(X_train, y_train)
```

Full coverage of GridSearchCV, RandomizedSearchCV, and cross-validation comes in Part 4.

Part 2 Recap — What You Now Know

- Three families of feature selection: filter (VarianceThreshold, SelectKBest with f_classif/f_regression/chi2/mutual_info), wrapper (RFE, RFECV), embedded (SelectFromModel + Lasso).
- PCA: dimensionality reduction by maximum variance. Always scale first. explained_variance_ratio_ tells you how much each component captures.
- Pipeline: chains preprocessing + model into one object, prevents data leakage.
- ColumnTransformer: parallel preprocessing on different columns. FeatureUnion: parallel transformers on the same data, outputs concatenated.
- LinearRegression: fits with normal equation, no gradient descent. Get coef_ and intercept_.
- Regression metrics: MAE, MSE, RMSE, R^2 , adjusted R^2 . Know how to compute R^2 by hand.
- PolynomialFeatures + LinearRegression = curved fits. Watch out for combinatorial explosion.
- Ridge (L2 - shrinks but doesn't zero), Lasso (L1 - zeros, selects features), ElasticNet (mix).
- SGDRegressor: gradient-descent-based. Use for big data. warm_start, partial_fit, mandatory scaling.

Self-check questions

1. You have 50 features. After `PCA(n_components=2).fit_transform(X)`, what's the new shape?
2. Which regularization can produce coefficients that are exactly zero — Ridge or Lasso?
3. What does `step__param` notation do in a Pipeline?
4. If `pca.explained_variance_ratio_ = [0.6, 0.25, 0.1]`, how much variance do you keep with `n_components=2`?
5. You want different preprocessing for numeric and categorical columns. What sklearn class?
6. What's the formula for R^2 and what range can it take?
7. SGDRegressor with `penalty='l1'` and `loss='squared_error'` is equivalent to which other model?
8. For a Ridge model with `param_grid = {'alpha': [0.1, 1, 10, 100]}` and 5-fold CV, how many fits?

Answers

1. **(n_rows, 2)** — PCA reduces columns to n_components.
2. **Lasso (L1)**. Ridge shrinks them but keeps them non-zero.
3. Lets you set the parameter **param** of pipeline step **step**. E.g., **'model__C'** sets C of the step named 'model'.
4. **0.85** (60% + 25%).
5. **ColumnTransformer**.
6. **$R^2 = 1 - \text{SS_res}/\text{SS_tot}$** . Range: $-\infty$ to 1.0. (1 = perfect, 0 = predicts mean, <0 = worse than mean.)
7. **Lasso** (L1 penalty + squared error loss = Lasso).

8. $4 \text{ alphas} \times 5 \text{ folds} = \mathbf{20 \text{ fits}}$. (After CV, GridSearchCV refits the best on all training data, so 21 total — but the standard answer is 20.)

What's Next — Part 3

Part 3 covers the heart of supervised learning — classification:

- **Logistic Regression** (binary, multinomial, OvR/OvO)
- **Classification metrics** — confusion matrix, precision, recall, F1, ROC-AUC (asked every paper!)
- **K-Nearest Neighbors**
- **Naive Bayes** (Gaussian, Multinomial, Bernoulli)
- **Support Vector Machines** (linear, kernels)
- **Decision Trees** (Gini, entropy, max_depth)
- **Ensemble methods** — Voting, Bagging, RandomForest, AdaBoost, GradientBoosting
- **MLP / Perceptron** (the basics)

When ready, just say "Part 3" and I'll build it.

MLP Exam Prep

Concept Mastery Guide — Part 3 of 4

Classification: Models & Metrics

Classification is roughly half of every PYQ. This part covers all the classifiers in your syllabus, the metrics used to evaluate them (this section alone is the heaviest in the exam), and the ensemble methods that combine multiple models. Take your time with the metrics — the confusion matrix questions are guaranteed marks.

What's in Part 3

#	Section	Why it matters
23	Logistic Regression	L4.6, L6.6 - asked every paper
24	Classification Metrics	L7.2-7.7 - HEAVIEST topic in PYQs
25	Multi-class strategies (OvR, OvO)	L4.6 - common
26	K-Nearest Neighbors	L4.8, L5.6, L6.11
27	Naive Bayes (Gaussian, Multinomial, Bernoulli)	L4.7, L6.9
28	Support Vector Machines (SVM)	L4.9, L6.12
29	Decision Trees	L4.10, L5.7, L6.13-6.14
30	Ensemble methods - Voting & Bagging	L4.11, L5.8, L6.15
31	Random Forest	Most popular ensemble
32	Boosting: AdaBoost & GradientBoosting	L4.12, L5.9, L6.16
33	Multi-layer Perceptron (MLP)	L4.13, L6.17

23. Logistic Regression

Despite the name, **logistic regression is a classification algorithm**, not a regression algorithm. It predicts the probability that a sample belongs to a class. It's the simplest classifier, and a great default to try first.

23.1 The intuition

Linear regression predicts any real number: -100, 5, 1500, anything. For classification, we need an output between 0 and 1 (a probability). Logistic regression takes a linear combination $z = \mathbf{w}_1 \mathbf{x}_1 + \mathbf{w}_2 \mathbf{x}_2 + \dots + \mathbf{b}$ and squashes it through the **sigmoid function**:

$$p = \text{sigmoid}(z) = 1 / (1 + e^{-z})$$

The sigmoid is an S-curve. It takes any real input and outputs a value between 0 and 1. Big positive $z \rightarrow$ close to 1. Big negative $z \rightarrow$ close to 0. $z = 0 \rightarrow$ exactly 0.5.

Decision rule

After computing the probability, you classify by thresholding (default 0.5):

- If $p \geq 0.5$, predict class 1 (positive).
- If $p < 0.5$, predict class 0 (negative).
- You can change this threshold to trade off precision and recall.

23.2 LogisticRegression in scikit-learn

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(
    penalty='l2',          # 'l1', 'l2', 'elasticnet', or None
    C=1.0,                 # INVERSE of regularization strength!
    solver='lbfgs',        # the optimization algorithm
    max_iter=100,
    multi_class='auto',    # auto-picks based on data
    random_state=42
)
model.fit(X_train, y_train)

model.coef_               # shape (n_classes, n_features) for multi-class
model.intercept_          # shape (n_classes,)
model.predict(X_test)     # hard predictions (class labels)
model.predict_proba(X_test) # probability for each class
model.score(X_test, y_test) # accuracy
```

Critical parameters

- **C**: the INVERSE of regularization strength. **Smaller C = stronger regularization**. This is opposite to alpha in Ridge/Lasso. C=1.0 is the default.
- **penalty**: 'l2' (default, Ridge-style), 'l1' (Lasso-style, can zero out coefficients), 'elasticnet' (mix), or None.
- **solver**: optimization algorithm. 'lbfgs' (default), 'liblinear' (good for small datasets & L1), 'saga' (large datasets, supports all penalties), 'newton-cg', 'sag'.
- **max_iter**: maximum iterations of the solver. Increase if you see ConvergenceWarning.

- **multi_class**: 'ovr' (one-vs-rest), 'multinomial' (softmax), or 'auto'.

WATCH OUT: The C confusion: In logistic regression, **larger C = LESS regularization = the model can fit more freely**. In Ridge/Lasso, **larger alpha = MORE regularization**. These are opposite. PYQs love this trap.

23.3 What logistic regression IS and ISN'T

Property	Truth
Is it a classifier?	Yes
Is it linear?	Yes — the decision boundary is linear (a line/plane in feature space)
Outputs probabilities?	Yes (predict_proba)
Needs feature scaling?	Yes, when using regularization (so coefficients are penalized fairly)
Sensitive to outliers?	Less than linear regression, but still affected
Handles multi-class?	Yes, via OvR or multinomial
Solves with gradient descent?	Yes (no closed-form like Linear Regression)

NOTE: Common PYQ: 'Which of these are TRUE about logistic regression?' Answers usually include: outputs probabilities, decision boundary is linear, can be regularized with L1/L2. Distractors: 'fits a curve' (no — linear boundary even though it uses sigmoid), 'minimizes MSE' (no — minimizes log-loss / cross-entropy).

24. Classification Metrics

This is the most important section in your exam. Confusion matrix, precision, recall, F1, ROC-AUC — these appear in EVERY paper, often multiple times. Master this and you'll bank a lot of marks.

24.1 The Confusion Matrix

For binary classification (positive class = 1, negative class = 0), the confusion matrix is a 2x2 table comparing predictions to actual labels:

	Predicted 0 (Negative)	Predicted 1 (Positive)
Actual 0 (Negative)	TN (True Negative)	FP (False Positive)
Actual 1 (Positive)	FN (False Negative)	TP (True Positive)

The four cells:

- **True Positive (TP):** actually 1, predicted 1 — correct positive prediction.
- **True Negative (TN):** actually 0, predicted 0 — correct negative prediction.
- **False Positive (FP):** actually 0, predicted 1 — "false alarm" / Type I error.
- **False Negative (FN):** actually 1, predicted 0 — "miss" / Type II error.

WATCH OUT: Sklearn convention: rows = actual, columns = predicted. So in sklearn's `confusion_matrix(y_true, y_pred)`, `cm[0][0]` = TN, `cm[0][1]` = FP, `cm[1][0]` = FN, `cm[1][1]` = TP. Some books flip this (rows=predicted), so always verify.

```
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_true, y_pred)
# array([[TN, FP],
#        [FN, TP]])
```

24.2 Accuracy

$$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$$

"What fraction of all predictions did I get right?"

```
from sklearn.metrics import accuracy_score
acc = accuracy_score(y_true, y_pred)
```

WATCH OUT: Accuracy is misleading for imbalanced data. If 99% of your data is class 0 and your model predicts 0 for everything, accuracy is 99% — but the model is useless. Always check precision/recall/F1 for imbalanced datasets.

24.3 Precision

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

"Of all the items I PREDICTED as positive, how many were actually positive?" Measures the quality of positive predictions. **High precision = few false alarms.**

Use when false positives are expensive. Example: spam filter — flagging an important email as spam (FP) is bad. You want high precision.

24.4 Recall (Sensitivity, True Positive Rate)

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

"Of all the items that ARE actually positive, how many did I catch?" Measures coverage. **High recall = few misses.**

Use when false negatives are expensive. Example: cancer screening — missing a real cancer (FN) is dangerous. You want high recall.

24.5 F1 Score — the harmonic mean

$$\text{F1} = 2 \cdot (\text{Precision} \cdot \text{Recall}) / (\text{Precision} + \text{Recall})$$

Combines precision and recall into one number. The **harmonic mean** punishes extreme values: if either precision or recall is very low, F1 is low. Useful when you want balanced performance.

```
from sklearn.metrics import precision_score, recall_score, f1_score, classification_report

precision = precision_score(y_true, y_pred)
recall = recall_score(y_true, y_pred)
f1 = f1_score(y_true, y_pred)

# Get all metrics in one report:
print(classification_report(y_true, y_pred))
```

NOTE: Worked example (memorize this):

Suppose: TP=40, FP=10, FN=20, TN=30.

- Accuracy = $(40+30)/100 = 0.70$
- Precision = $40/(40+10) = 0.80$
- Recall = $40/(40+20) = 0.667$
- F1 = $2 \times (0.8 \times 0.667) / (0.8 + 0.667) = 0.727$

24. Classification Metrics (continued)

24.6 Specificity (True Negative Rate)

$$\text{Specificity} = \text{TN} / (\text{TN} + \text{FP})$$

"Of all the items that ARE actually negative, how many did I correctly identify as negative?" Less common in sklearn but appears in PYQ theoretical questions.

24.7 ROC Curve and AUC

Classifiers like LogisticRegression don't just give you a class — they give you a probability. The default threshold of 0.5 isn't sacred. As you change the threshold, your TP rate (recall) and FP rate change. The **ROC curve** plots them against each other for all possible thresholds:

- X-axis: **False Positive Rate (FPR) = FP / (FP + TN)**
- Y-axis: **True Positive Rate (TPR) = Recall = TP / (TP + FN)**

Each point on the ROC curve represents one threshold choice. The closer the curve hugs the top-left corner, the better the model. A diagonal line is random guessing.

AUC — Area Under the (ROC) Curve

AUC is a single number summarizing the entire ROC curve:

- **AUC = 1.0**: perfect classifier.
- **AUC = 0.5**: random guessing.
- **AUC < 0.5**: worse than random (your labels might be flipped!)

AUC has a nice interpretation: it's the probability that the model ranks a randomly chosen positive sample higher than a randomly chosen negative sample.

```
from sklearn.metrics import roc_auc_score, roc_curve

# Need probability scores, not class labels:
y_proba = model.predict_proba(X_test)[: , 1] # probability of class 1
auc = roc_auc_score(y_test, y_proba)

# Get the actual ROC curve points:
fpr, tpr, thresholds = roc_curve(y_test, y_proba)
```

TIP: AUC vs Accuracy: AUC is threshold-independent (uses all thresholds). Accuracy uses only the 0.5 threshold. AUC is more informative for imbalanced data and for comparing models.

24.8 Multi-class metrics — averaging strategies

For multi-class classification, precision/recall/F1 are computed per class and then averaged. There are three averaging strategies:

Strategy	How it works	Use when
'micro'	Aggregate TP/FP/FN across all classes, then compute the metric once	All samples matter equally
'macro'	Compute per-class, then take an unweighted mean	All classes matter equally (regardless of size)

Strategy	How it works	Use when
'weighted'	Compute per-class, then take a mean weighted by class support (size)	Imbalanced classes; bigger classes matter more
None	Return per-class array (no averaging)	You want to inspect each class

```
f1_score(y_true, y_pred, average='macro')    # mean of per-class F1
f1_score(y_true, y_pred, average='weighted') # weighted by class size
f1_score(y_true, y_pred, average='micro')    # global TP/FP/FN, single F1
f1_score(y_true, y_pred, average=None)       # array, one per class
```

NOTE: Important fact: for multi-class, **micro-averaged precision = micro-averaged recall = micro-averaged F1 = accuracy**. They all reduce to the same number when every sample has exactly one prediction.

24.9 The assertion-reason PYQ pattern

A frequent PYQ format: "Assertion (A): This is a good model with 96% accuracy. Reason (R): For imbalanced data, accuracy may not be a good metric."

If 96% of the data is one class, a dummy model that always predicts that class gets 96% accuracy without learning anything. So **both A and R are correct, but R does NOT explain A — in fact R EXPLAINS WHY A IS WRONG**. The trick is reading carefully.

25. Multi-class Strategies (OvR & OvO)

Some classifiers (logistic regression, SVM) are inherently binary. To handle 3+ classes, they use one of two strategies under the hood.

25.1 One-vs-Rest (OvR), also called One-vs-All

Train **n binary classifiers**, one per class. Each one learns to separate "class i" from "everything else." To predict, run all n models and pick the class with the highest confidence score.

- 3-class problem → 3 binary classifiers.
- Fast: only n classifiers.
- Each classifier sees the entire dataset (one class as positive, all others as negative).

25.2 One-vs-One (OvO)

Train **$n \times (n-1) / 2$ binary classifiers**, one for each pair of classes. To predict, run all classifiers and use majority voting.

- 3 classes → 3 classifiers (1-vs-2, 1-vs-3, 2-vs-3).
- 4 classes → 6 classifiers.
- 10 classes → 45 classifiers.
- Each classifier only sees data from its 2 classes (smaller training set per model).

25.3 Multinomial (Softmax) Logistic Regression

Logistic regression has a third option: train ONE model that natively predicts probabilities for all classes simultaneously, using the **softmax function** instead of sigmoid. This is **multinomial logistic regression** (a.k.a. softmax regression).

$$p_i = e^{z_i} / \sum_j e^{z_j}$$

Each class gets its own linear score z_i . The softmax converts those scores into probabilities that sum to 1 across all classes.

```
LogisticRegression(multi_class='ovr')           # one-vs-rest
LogisticRegression(multi_class='multinomial')    # softmax
LogisticRegression(multi_class='auto')          # picks based on solver
```


25.4 OvR vs OvO — comparison

	OvR (One-vs-Rest)	OvO (One-vs-One)
Number of classifiers	n	$n \times (n-1) / 2$
Training set per classifier	Entire dataset	Just two classes' data
Per-classifier training time	Slow (full data)	Fast (subset)
Total training time	Usually faster	Usually slower
Class imbalance issues?	Yes — one vs many is unbalanced	No — pairs are balanced
Used by	LogisticRegression default for some solvers	SVM (always uses OvO)

NOTE: SVM always uses OvO internally in sklearn (because OvR with SVM has scaling issues). LogisticRegression defaults to OvR or multinomial depending on the solver.

26. K-Nearest Neighbors (KNN)

KNN is the simplest possible classifier: "to predict a new point, find the k nearest training points and let them vote." It does **NO training** — it just memorizes the training data and does all the work at prediction time. This is called a **lazy learner** or **instance-based learner**.

26.1 The algorithm

- 1. Memorize all training points ($X_{\text{train}}, y_{\text{train}}$).
- 2. For a new query point x_{new} :
 - a. Compute distance from x_{new} to every training point.
 - b. Find the k closest training points.
 - c. For classification: majority vote among their labels. For regression: average their values.

26.2 KNN in scikit-learn

```
from sklearn.neighbors import KNeighborsClassifier, KNeighborsRegressor
```

```
clf = KNeighborsClassifier(  
    n_neighbors=5,          # the 'k' value  
    weights='uniform',      # 'uniform' or 'distance'  
    algorithm='auto',       # 'ball_tree', 'kd_tree', 'brute', 'auto'  
    metric='minkowski',     # default; with p=2 = Euclidean  
    p=2                     # 1 = Manhattan, 2 = Euclidean  
)  
clf.fit(X_train, y_train)   # this just stores the data!  
clf.predict(X_test)         # this is where all the work happens
```

Critical parameters

- **n_neighbors (k)**: the most important. Small k = noisy, sensitive to outliers. Large k = oversmoothed, may miss local patterns. Common starting point: **k = sqrt(n)**.
- **weights='uniform'** (default): all k neighbors get equal vote. **'distance'**: closer neighbors get bigger vote (weight = $1/\text{distance}$).
- **metric**: how to measure distance. 'minkowski' is generic; with p=1 it's Manhattan, p=2 it's Euclidean.
- **algorithm**: how to find neighbors fast. 'brute' (compare to every point), 'kd_tree' / 'ball_tree' (spatial indexes), 'auto' picks for you.

26.3 KNN's strengths and weaknesses

Strengths	Weaknesses
No training time (just stores data)	Slow at prediction time ($O(n)$ per query)
Naturally handles multi-class	Requires lots of memory (stores all data)
No assumptions about data shape	Suffers in high dimensions (curse of dimensionality)
Can capture complex non-linear patterns	Sensitive to feature scale — MUST scale!
Easy to understand	Sensitive to irrelevant features

WATCH OUT: The single most-asked KNN question: What is the primary computational bottleneck of KNN? Answer: **storing and searching through all training samples at prediction time** — not training. Training is essentially free.

26.4 Why feature scaling is mandatory

KNN computes distances between points. If feature 1 ranges 0-1 and feature 2 ranges 0-1,000,000, then feature 2 dominates every distance computation — feature 1 becomes irrelevant. **Always StandardScaler or MinMaxScaler before KNN.**

26.5 The curse of dimensionality

In high-dimensional space, distances stop being meaningful — all points become roughly equidistant from each other. KNN's notion of "nearest neighbors" breaks down. Practical limit: KNN works well up to ~10-20 features. Beyond that, use PCA first or pick a different algorithm.

27. Naive Bayes

Naive Bayes is a probabilistic classifier based on **Bayes' theorem** with one strong (and rarely true) assumption: **features are independent given the class**. Hence "naive." Despite this naive assumption, it works surprisingly well in practice, especially for text classification.

27.1 Bayes' theorem (the heart of it)

$$P(\text{class} \mid \text{features}) = P(\text{features} \mid \text{class}) \cdot P(\text{class}) / P(\text{features})$$

We want $P(\text{class} \mid \text{features})$ — "given this email, how likely is it spam?" Bayes lets us compute it from quantities easier to estimate from training data: **$P(\text{features} \mid \text{class})$** (likelihood) and **$P(\text{class})$** (prior).

The "naive" part: assume features are independent of each other given the class, so **$P(\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n \mid \text{class}) = P(\mathbf{x}_1 \mid \text{class}) \cdot P(\mathbf{x}_2 \mid \text{class}) \cdot \dots \cdot P(\mathbf{x}_n \mid \text{class})$** . This makes the math easy and fast.

27.2 Three flavors in scikit-learn

Variant	Use for	Assumes features are...
GaussianNB	Continuous numeric features	Normally distributed within each class
MultinomialNB	Discrete counts (word counts, n-grams)	Multinomial — non-negative integers
BernoulliNB	Binary features (0/1, presence/absence)	Bernoulli (0 or 1)
ComplementNB	Imbalanced text data	Variant of MultinomialNB
CategoricalNB	Discrete categorical features	Each feature has a finite set of categories

27.3 GaussianNB — for continuous features

```
from sklearn.naive_bayes import GaussianNB

clf = GaussianNB()
clf.fit(X_train, y_train)

clf.theta_      # mean of each feature for each class
clf.var_        # variance of each feature for each class
clf.class_prior_ # P(class) computed from training data
```

GaussianNB models each feature within each class as a normal distribution. To predict, it computes how likely the new point is under each class's distribution, multiplies by the class prior, and picks the class with highest posterior.

27.4 MultinomialNB — for text/counts

```
from sklearn.naive_bayes import MultinomialNB

clf = MultinomialNB(alpha=1.0) # alpha = Laplace smoothing
clf.fit(X_train_counts, y_train)
```

alpha is the **Laplace (additive) smoothing** parameter. Without it, if a word in your test set never appeared with a particular class in training, the predicted probability becomes zero. Adding alpha

(default 1.0) avoids this. Setting $\alpha=0$ disables smoothing.

27.5 Naive Bayes properties

- **Very fast** — both training and prediction. Just count and multiply.
- **Works well with small data** — the strong independence assumption acts as a regularizer.
- **Handles multi-class natively** — no OvR/OvO needed.
- **NOT impacted by feature scaling** — uses probabilities, not distances.
- **Can struggle when features are correlated** — the independence assumption is violated.
- **Doesn't learn feature interactions** — treats every feature independently.

NOTE: Famous Naive Bayes use case: spam classification. The bag-of-words representation treats each word as a feature, and MultinomialNB on word counts works very well — this is essentially how early spam filters worked.

28. Support Vector Machines (SVM)

SVM tries to find the **best line (or curve) that separates the classes**. "Best" means: maximizes the margin, i.e., the gap between the line and the nearest points of either class. Those nearest points are called **support vectors** — only they matter for the decision.

28.1 The intuition

Imagine 2 classes of points on a 2D plane. There are infinitely many lines that could separate them. SVM picks the line with the **maximum margin** — the line that has the widest "street" between the two classes. This tends to generalize better than just any separating line.

28.2 Hard-margin vs soft-margin

Hard-margin SVM: requires perfect separation. No point can be on the wrong side or inside the margin. Only works if classes are perfectly separable, which is rare.

Soft-margin SVM: allows some points to be on the wrong side or inside the margin, but penalizes them. The penalty strength is controlled by **C**:

- **Large C:** heavy penalty → tries hard to classify all points correctly → risks overfitting.
- **Small C:** lenient → allows more violations → smoother boundary, less overfitting.

28.3 The kernel trick

SVM finds linear boundaries in feature space. For non-linear problems, we transform the data into a higher-dimensional space where it IS linearly separable. The **kernel trick** does this implicitly — we never explicitly compute the higher-dimensional features, just kernel function values.

Kernel	Formula	Use for
'linear'	$K(x, y) = x \cdot y$	When data is linearly separable; high dimensions like text
'poly'	$K(x, y) = (\text{gamma} \cdot x \cdot y + \text{coef0})^{\text{degree}}$	Polynomial decision boundaries
'rbf' (default)	$K(x, y) = \exp(-\text{gamma} \cdot x - y ^2)$	General-purpose, smooth non-linear boundaries
'sigmoid'	$K(x, y) = \tanh(\text{gamma} \cdot x \cdot y + \text{coef0})$	Like a neural network; less common

28.4 SVM in scikit-learn

```
from sklearn.svm import SVC, SVR, LinearSVC

clf = SVC(
    C=1.0,          # regularization (smaller C = stronger regularization)
    kernel='rbf',    # 'linear', 'poly', 'rbf', 'sigmoid'
    gamma='scale',   # kernel coefficient ('scale', 'auto', or float)
    degree=3,        # for poly kernel only
    probability=False # if True, enables predict_proba (slow)
)
clf.fit(X_train, y_train)

clf.support_vectors_ # the actual support vector points
clf.n_support_       # how many support vectors per class
```

Critical parameters

- **C**: like in logistic regression, smaller = more regularization.
- **kernel**: which kernel to use. RBF is the default and a strong general choice.
- **gamma**: for RBF/poly/sigmoid. '**scale**' = $1/(n_features \cdot X.var())$, '**auto**' = $1/n_features$. Large gamma = tight, complex boundary; small gamma = smooth boundary.
- **probability=True**: enables predict_proba via internal cross-validation (5-fold). Adds a lot of training time.

WATCH OUT: SVM is sensitive to feature scaling. Always scale your features before SVM. Without scaling, large-magnitude features dominate the distance calculation in the kernel.

28.5 LinearSVC vs SVC(kernel='linear')

Both fit linear SVMs but use different solvers. **LinearSVC** is faster on large datasets and supports L1 penalty. **SVC(kernel='linear')** uses the standard libsvm library and supports OvO multi-class. Use LinearSVC for big data, SVC for flexibility.

29. Decision Trees

A decision tree is a flowchart of yes/no questions about your features. "Is age > 30?" If yes, go right. "Is income > 50K?" Each leaf node makes a final prediction. They're intuitive, interpretable, and the building block for Random Forest and gradient boosting.

29.1 How a tree is built

- 1. At each node, find the FEATURE and THRESHOLD that best splits the data into purer groups.
- 2. Apply that split, dividing the data into left and right children.
- 3. Recurse: repeat the process on each child.
- 4. Stop when a stopping criterion is met (max depth reached, leaf is pure enough, too few samples).
- 5. At each leaf, the prediction is the majority class (classification) or the mean (regression).

29.2 Splitting criteria — Gini and Entropy

How do we decide which feature/threshold gives the "purest" split? Two common measures of impurity for classification:

Gini Impurity: probability of misclassifying a randomly drawn sample if you labeled it according to the class distribution at the node. $\text{Gini} = 1 - \sum p_i^2$

Entropy: measure of disorder/uncertainty. $\text{Entropy} = -\sum p_i \cdot \log_2(p_i)$

- Both are 0 when the node is pure (all one class).
- Both are maximum when classes are equally mixed.
- Gini is faster (no logarithm); they usually give similar trees.
- For regression, sklearn uses MSE or MAE as the splitting criterion.

29.3 DecisionTreeClassifier in sklearn

```
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor

clf = DecisionTreeClassifier(
    criterion='gini',          # 'gini' (default) or 'entropy' or 'log_loss'
    max_depth=None,           # how deep the tree can grow
    min_samples_split=2,      # min samples needed to split a node
    min_samples_leaf=1,       # min samples required at a leaf
    max_features=None,        # how many features to consider at each split
    random_state=42
)
clf.fit(X_train, y_train)

clf.feature_importances_      # how important each feature was
clf.tree_.node_count          # how many nodes in the tree
```

Hyperparameters that control overfitting

- **max_depth:** maximum depth. None = grow until leaves are pure (will overfit).
- **min_samples_split:** minimum samples in a node before it can be split. Higher = simpler tree.

- **min_samples_leaf**: minimum samples that must end up in a leaf. Higher = simpler tree, smoother predictions.
- **max_leaf_nodes**: cap on total number of leaves.
- **max_features**: number of features randomly considered at each split. Used heavily by RandomForest.
- **ccp_alpha**: cost-complexity pruning parameter for post-pruning. Higher = more aggressive pruning.

29.4 Decision tree properties

Strengths	Weaknesses
No need for feature scaling (uses splits, not distances)	Easily overfits if not pruned
Handles numeric & categorical features	Unstable: small data changes give very different trees
Handles missing values to some extent	Greedy: locally optimal splits, not global optimum
Interpretable (you can draw the tree)	Decision boundaries are axis-aligned (rectangles)
Naturally multi-class	Single tree often weaker than ensembles

NOTE: Trees don't need feature scaling. A split is just "is feature X > threshold?" Whether that threshold is 5 or 5,000,000 makes no difference to the tree's behavior.

30. Ensemble Methods — Voting & Bagging

An ensemble combines multiple models to get better predictions than any single model. There are two main philosophies: **parallel** (train models independently, combine their predictions) and **sequential** (each new model fixes the previous one's mistakes — that's boosting, covered in the next section).

30.1 VotingClassifier

Train several DIFFERENT models on the same data, then combine their predictions:

- **voting='hard'**: each model votes for a class, majority wins.
- **voting='soft'**: average the predicted probabilities, pick class with highest average.

```
from sklearn.ensemble import VotingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC

clf = VotingClassifier(
    estimators=[
        ('lr', LogisticRegression()),
        ('dt', DecisionTreeClassifier()),
        ('svc', SVC(probability=True)) # need probability=True for soft
    ],
    voting='soft'
)
clf.fit(X_train, y_train)
```

NOTE: Hard vs Soft voting: Soft voting usually performs better because it uses confidence information, but requires all models to support `predict_proba`. **Soft voting averages probabilities — it does NOT pick the class with highest sum of probabilities** (a common PYQ distractor).

30.2 Bagging (Bootstrap Aggregating)

Train MANY copies of the SAME model on different bootstrap samples (random samples with replacement) of the training data. Then combine their predictions by voting (classification) or averaging (regression).

Why does bagging help?

Each individual model might overfit to the noise in its bootstrap sample. But the noise is different across samples, so when you average their predictions, the noise cancels out while the true signal stays. Bagging reduces **variance** without increasing bias. Especially effective for high-variance models like deep decision trees.

```
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier

clf = BaggingClassifier(
    estimator=DecisionTreeClassifier(), # the base model
    n_estimators=100,                   # how many to train
    max_samples=1.0,                    # fraction of training samples per model
    max_features=1.0,                   # fraction of features per model
    bootstrap=True,                     # sample WITH replacement
    bootstrap_features=False,            # sample features without replacement
)
```

```
oob_score=True,                # use out-of-bag samples for validation
n_jobs=-1,                     # parallel training
random_state=42
)
clf.fit(X_train, y_train)
clf.oob_score_  # validation score from out-of-bag samples
```

Out-of-bag (OOB) score

When you bootstrap-sample a dataset, on average ~37% of samples are NOT picked. These **out-of-bag** samples weren't seen by that particular tree, so we can use them as a free validation set. Set **oob_score=True** to get this estimate — it's like free cross-validation.

30.3 Pasting — sampling without replacement

If you set **bootstrap=False**, BaggingClassifier samples WITHOUT replacement — this variant is called **pasting**. It's used less often than bagging.

31. Random Forest

Random Forest is bagging + decision trees + an extra trick: **at each split, the tree only considers a random subset of features**. This decorrelates the trees and gives a much better ensemble than plain bagging of trees.

31.1 Why the extra randomness?

In plain bagging of trees, all trees still have access to all features, so they tend to make similar splits (especially picking the strongest features at the top). The trees are correlated. Adding feature randomness at each split forces trees to be different, which makes the ensemble's variance reduction much stronger.

31.2 RandomForestClassifier in sklearn

```
from sklearn.ensemble import RandomForestClassifier, RandomForestRegressor

clf = RandomForestClassifier(
    n_estimators=100,          # number of trees
    criterion='gini',          # or 'entropy'
    max_depth=None,
    min_samples_split=2,
    min_samples_leaf=1,
    max_features='sqrt',       # 'sqrt' for classification, 1.0 for regression
    bootstrap=True,
    oob_score=False,
    n_jobs=-1,
    random_state=42
)
clf.fit(X_train, y_train)

clf.feature_importances_      # average importance across all trees
clf.estimators_                # list of individual trees
```

Critical parameters

- **n_estimators**: number of trees. More is better (up to a point), at the cost of training time.
- **max_features**: number of features randomly chosen at each split. **'sqrt'** is the default and works well for classification. **'log2'** also common. **1.0** = use all features (no randomness, just bagging).
- **max_depth**: usually leave at None (let trees grow deep) since the ensemble averages them out.
- **bootstrap=True**: enables sampling with replacement (the bagging part).

WATCH OUT: PYQ trap: 'min_samples_leaf controls the maximum number of trees that will be created.' This is FALSE. min_samples_leaf is the minimum samples in a leaf; **n_estimators** controls the number of trees. The exam loves planting these distractors.

31.3 Random Forest properties

- **Reduces overfitting** compared to single trees by averaging.
- **No need for scaling** — tree-based.
- **Naturally handles multi-class**.
- **Provides feature importances** for free.
- **Embarrassingly parallel** — trees train independently.

- **OOB score** available for free if `oob_score=True`.
- **Less interpretable** than a single tree.
- **Slower** than a single tree, both for training and prediction.

31.4 Random Forest vs single Decision Tree

	Decision Tree	Random Forest
Number of trees	1	Many (100+)
Variance	High (overfits easily)	Low (averaged out)
Interpretability	High (one flowchart)	Low (forest of 100 trees)
Training speed	Fast	Slower (<code>n_estimators</code> times)
Typical accuracy	Moderate	High
When to use	Need interpretability	Want best accuracy without tuning much

32. Boosting: AdaBoost & Gradient Boosting

Boosting is the OTHER major ensemble philosophy: train models **sequentially**, with each new model focusing on the mistakes of the previous ones. Done right, this turns many **weak learners** (slightly better than random) into one **strong learner**.

32.1 AdaBoost (Adaptive Boosting)

AdaBoost trains weak classifiers (usually decision tree stumps — depth-1 trees) one at a time:

- 1. Start with equal weight on every training sample.
- 2. Train a weak classifier on the weighted data.
- 3. Increase the weight of misclassified samples (so the next model focuses on them).
- 4. Decrease the weight of correctly classified samples.
- 5. Add the new model to the ensemble with a weight proportional to its accuracy.
- 6. Repeat for `n_estimators` rounds.

Final prediction: weighted vote of all the weak classifiers.

```
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier

clf = AdaBoostClassifier(
    estimator=DecisionTreeClassifier(max_depth=1), # weak learner = stump
    n_estimators=50,
    learning_rate=1.0, # how much each new model contributes
    random_state=42
)
clf.fit(X_train, y_train)
```

32.2 Gradient Boosting

Gradient Boosting is more general and more powerful than AdaBoost. Instead of reweighting samples, each new model tries to predict the **residual errors** of the current ensemble. The idea: the ensemble's prediction is the sum of all models, and each new model is a small step in the direction that reduces the loss.

Algorithm sketch:

- 1. Initialize prediction as the mean of `y` (for regression).
- 2. Compute residuals: `r = y - current_prediction`.
- 3. Train a small tree to predict the residuals.
- 4. Add `learning_rate × tree's prediction` to the current prediction.
- 5. Repeat for `n_estimators` rounds.

```
from sklearn.ensemble import GradientBoostingClassifier, GradientBoostingRegressor

clf = GradientBoostingClassifier(
    n_estimators=100,
    learning_rate=0.1, # smaller = need more estimators, but more accurate
    max_depth=3, # weak learners (depth 3 trees)
    subsample=1.0, # fraction of samples per tree (=Stochastic GB if <1)
    random_state=42
)
clf.fit(X_train, y_train)
```

The learning_rate / n_estimators tradeoff

learning_rate shrinks each tree's contribution. **Smaller learning_rate** means you need **more n_estimators** to fit the data, but the result generalizes better. Common combination: **learning_rate=0.1, n_estimators=100**.

32.3 Bagging vs Boosting — the comparison every PYQ asks

	Bagging (incl. Random Forest)	Boosting (AdaBoost, GB)
How models train	Independently, in parallel	Sequentially, each fixes prior mistakes
Sample weights	All samples equal	Misclassified get higher weights (AdaBoost)
Reduces	Variance (overfitting)	Bias (underfitting)
Each model is	Strong (deep trees)	Weak (shallow trees, stumps)
Prediction	Average / majority vote	Weighted vote / sum
Risk	Less prone to overfitting	Can overfit if too many estimators
Parallelizable?	Yes (trees independent)	No (sequential dependency)

NOTE: The cleanest one-line summary: **Bagging reduces variance, boosting reduces bias.** Random Forest is parallel; AdaBoost and GradientBoosting are sequential.

33. Multi-layer Perceptron (MLP)

The MLP is the simplest feedforward neural network. It's a stack of layers: an input layer (one node per feature), one or more **hidden layers**, and an output layer. Each layer applies a linear combination followed by a non-linear **activation function**. This non-linearity is what lets MLPs capture complex patterns that linear models can't.

33.1 The Perceptron — the simplest case

A single Perceptron is just **step($w \cdot x + b$)** — a linear classifier with a step activation. It can only learn LINEAR decision boundaries. The famous failure: a single Perceptron can't learn XOR (which is not linearly separable). To learn XOR you need at least one hidden layer — that's the MLP.

```
from sklearn.linear_model import Perceptron
clf = Perceptron(max_iter=1000, eta0=1.0, random_state=42)
clf.fit(X_train, y_train)
```

33.2 MLPClassifier and MLPRegressor

```
from sklearn.neural_network import MLPClassifier, MLPRegressor

clf = MLPClassifier(
    hidden_layer_sizes=(100,),      # one hidden layer with 100 neurons
    activation='relu',              # 'identity', 'logistic', 'tanh', 'relu'
    solver='adam',                  # 'lbfgs', 'sgd', 'adam'
    alpha=0.0001,                   # L2 regularization strength
    batch_size='auto',
    learning_rate='constant',        # 'constant', 'invscaling', 'adaptive'
    learning_rate_init=0.001,
    max_iter=200,
    random_state=42
)
clf.fit(X_train, y_train)
```

hidden_layer_sizes — the architecture

- **(100,)**: one hidden layer with 100 neurons.
- **(64, 32)**: two hidden layers with 64 then 32 neurons.
- **(50, 50, 50)**: three hidden layers, all with 50 neurons.

Activation functions

Activation	Formula	When to use
relu (default)	$\max(0, x)$	Default choice for hidden layers; fast and effective
tanh	$(e^x - e^{-x}) / (e^x + e^{-x})$	Output in $(-1, 1)$; zero-centered
logistic (sigmoid)	$1 / (1 + e^{-x})$	Output in $(0, 1)$; used in output layer for binary classification
identity	x	Linear (no activation); for regression output layer

33.3 Output layer is automatic in sklearn

Sklearn automatically adds an appropriate output layer:

- **MLPClassifier (binary)**: 1 output neuron with logistic (sigmoid) activation.
- **MLPClassifier (multi-class)**: n output neurons with softmax activation.
- **MLPRegressor**: 1 (or more) output neuron(s) with linear (identity) activation.

WATCH OUT: PYQ trap: 'For MLPRegressor predicting house prices, the output layer should use softmax.' FALSE. Softmax is for multi-class probability outputs. For regression, the output is identity (linear). Continuous prices need a continuous output.

33.4 MLP requirements

- **Feature scaling is mandatory.** Like SGD, gradient-based optimization needs scaled features.
- **Reproducibility:** set `random_state`, since weight initialization is random.
- **max_iter:** increase if you see `ConvergenceWarning`.
- **warm_start=True:** continue training from previous weights (same idea as `SGDRegressor`).

33.5 What MLPs CANNOT do (in sklearn)

Sklearn's MLP is a basic implementation. It does NOT support: convolutional layers (for images), recurrent layers (for sequences), GPUs, dropout, batch normalization, transfer learning. For those, use TensorFlow or PyTorch. Sklearn's MLP is fine for tabular data.

NOTE: PYQ tip: 'Increasing the number of hidden layers always improves accuracy.' FALSE. Deeper networks have more parameters and can overfit, especially on small datasets. More layers also makes training harder and slower.

Part 3 Recap — What You Now Know

- **Logistic Regression**: classifier despite the name. Sigmoid squashes linear scores into probabilities. C is INVERSE of regularization.
- **Confusion matrix**: TP, TN, FP, FN. Sklearn convention: rows=actual, cols=predicted.
- **Precision** = $TP/(TP+FP)$ (quality of positives), **Recall** = $TP/(TP+FN)$ (coverage of positives), **F1** = harmonic mean.
- **ROC-AUC**: threshold-independent metric. Plot TPR vs FPR. AUC=1 perfect, 0.5 random.
- **Multi-class averaging**: micro (=accuracy), macro (unweighted mean), weighted (by class size).
- **OvR vs OvO**: OvR = n classifiers, one per class. OvO = $n(n-1)/2$, one per pair. SVM uses OvO.
- **KNN**: lazy learner, no training. Bottleneck is prediction time. MUST scale features.
- **Naive Bayes**: assumes feature independence. GaussianNB (continuous), MultinomialNB (counts), BernoulliNB (binary). NO scaling needed.
- **SVM**: maximum margin, support vectors only. Kernels (linear, poly, rbf, sigmoid). C = inverse regularization. MUST scale.
- **Decision Trees**: greedy splits using Gini or Entropy. NO scaling needed. Easily overfit, control with max_depth, min_samples_*.
- **Voting**: combine different models. Hard = majority vote on classes; Soft = average probabilities.
- **Bagging**: same model + bootstrap samples. Reduces variance. OOB score is free CV.
- **Random Forest**: bagging + random feature subsets at each split. Great default classifier.
- **AdaBoost**: sequential weak learners (stumps), reweight misclassified samples.
- **Gradient Boosting**: each tree predicts residuals. learning_rate × n_estimators tradeoff.
- **MLP**: stacked layers + activations. ReLU default for hidden. Output layer is auto. MUST scale.

Self-check questions

1. Confusion matrix is $\begin{bmatrix} 80 & 5 \\ 10 & 25 \end{bmatrix}$. Compute precision, recall, F1, accuracy.
2. For 5-class classification with OvR, how many classifiers are trained? What about OvO?
3. You train an SVM with $C=0.001$. What does this imply?
4. Which of these don't need feature scaling: KNN, Decision Tree, SVM, Naive Bayes, MLP?
5. What's the bottleneck of KNN: training or prediction?
6. Bagging reduces ____, boosting reduces ____.
7. For a Perceptron, what does warm_start=True do?
8. You have AUC = 0.5. What does this mean?
9. Soft voting picks the class with the highest sum of probabilities. True or false?
10. RandomForest with max_features=1.0 is equivalent to plain ____ of decision trees.

Answers

1. TN=80, FP=5, FN=10, TP=25. Accuracy=(80+25)/120=0.875. Precision=25/(25+5)=0.833. Recall=25/(25+10)=0.714. F1=2×(0.833×0.714)/(0.833+0.714)=0.769.
2. OvR: **5 classifiers**. OvO: $5 \times 4 / 2 = 10$ classifiers.

3. Very **strong regularization** — the model will be very simple, likely underfit.
4. **Decision Tree, Random Forest, Naive Bayes** don't need scaling. KNN, SVM, MLP, Logistic Regression (with regularization) DO need scaling.
5. **Prediction** — KNN does almost no work during fit, all the work happens at predict time.
6. Bagging reduces **variance**, boosting reduces **bias**.
7. **Continues training from previous weights** instead of restarting on next fit() call. Same as SGDRegressor.
8. The model is **no better than random guessing**.
9. **FALSE**. Soft voting **averages** the probabilities (or sums them, then divides by n — mathematically equivalent for picking the max). The PYQ trap version says "sum" without explaining; check the wording.
10. Plain **bagging** of decision trees. With max_features=1.0 there's no extra randomness at splits, just the bootstrap sampling.

What's Next — Part 4 (Final)

Part 4 wraps everything up:

- **K-Means clustering** — centroids, inertia, k-means++
- **Hierarchical Agglomerative Clustering (HAC)** — linkages, dendrograms
- **Cross-validation** — KFold, StratifiedKFold, LeaveOneOut, ShuffleSplit
- **GridSearchCV & RandomizedSearchCV** in depth, with combination math
- **Transfer Learning** (in your syllabus)
- **Final exam cheat sheet** — one-page reference of every sklearn class & key parameter
- **Last-day revision strategy**

When ready, just say "Part 4" and I'll build it.

MLP Exam Prep

Concept Mastery Guide — Part 4 of 4 (Final)

Clustering, Tuning, Transfer Learning & Final Cheat Sheet

The final part. Unsupervised learning (clustering), the proper way to evaluate models (cross-validation), how to systematically tune hyperparameters, transfer learning, and a printable one-page cheat sheet that covers the whole course. Read this last, then revise.

What's in Part 4

#	Section	Why it matters
34	K-Means Clustering	L9.1 - asked every paper
35	Hierarchical Agglomerative Clustering (HAC)	L9.2 - heavy in 2025 papers
36	End-to-end ML project workflow	L9.4-9.8
37	Cross-validation in depth	Asked every paper
38	GridSearchCV & RandomizedSearchCV	L4.5 - combination math is tested
39	Transfer Learning	In your syllabus (Week 12)
40	Bias-Variance tradeoff & overfitting	Foundational concept
41	The Final Cheat Sheet	One-page exam reference
42	Last-day revision strategy	Day-of-exam plan

34. K-Means Clustering

K-Means is the most popular **unsupervised** algorithm. "Unsupervised" means there are no labels — you only have X, no y. The goal: discover natural groupings (clusters) in the data. K-Means partitions n samples into k groups, where each sample belongs to the cluster with the nearest **centroid** (cluster center).

34.1 The algorithm (Lloyd's algorithm)

- 1. Pick k initial centroids (randomly or smarter, see k-means++ below).
- 2. **Assignment step**: assign each point to the nearest centroid (Euclidean distance).
- 3. **Update step**: move each centroid to the mean of its assigned points.
- 4. Repeat steps 2-3 until centroids stop moving (convergence) or max_iter is reached.

K-Means minimizes **inertia** (also called within-cluster sum of squares, WCSS): the total squared distance from each point to its assigned centroid.

$$\text{Inertia} = \sum_i \sum_{\mathbf{x} \in C_i} \|\mathbf{x} - \mu_i\|^2$$

34.2 KMeans in scikit-learn

```
from sklearn.cluster import KMeans
```

```
km = KMeans(  
    n_clusters=3,          # k - the number of clusters  
    init='k-means++',      # 'k-means++' (smart) or 'random'  
    n_init=10,             # number of times to run with different initializations  
    max_iter=300,  
    tol=1e-4,  
    random_state=42  
)  
km.fit(X)
```

```
km.cluster_centers_  # shape (n_clusters, n_features) - the centroids  
km.labels_           # which cluster each training point belongs to  
km.inertia_          # the final WCSS (lower = tighter clusters)  
km.predict(X_new)    # assign new points to clusters
```

Critical parameters

- **n_clusters (k)**: you have to choose this BEFORE training. Use elbow method or silhouette score to pick (see below).
- **init='k-means++'** (default): smart initialization that spreads initial centroids out. Beats random initialization in nearly every case.
- **n_init**: K-Means is sensitive to initial centroids. sklearn runs the full algorithm n_init times with different inits and keeps the best result (lowest inertia). Default 10 in older versions, 'auto' in newer.
- **max_iter**: maximum iterations per run.
- **tol**: stop early if centroids barely move between iterations.

34.3 Choosing k — the elbow method

Plot inertia vs k for k = 1, 2, 3, ..., 10. Inertia always decreases as k grows (more clusters = tighter fit). But there's usually an "elbow" where inertia stops dropping sharply — that's a reasonable k.

```
inertias = []
for k in range(1, 11):
    km = KMeans(n_clusters=k, random_state=42).fit(X)
    inertias.append(km.inertia_)

# Plot inertias and look for the elbow:
import matplotlib.pyplot as plt
plt.plot(range(1, 11), inertias, 'o-')
plt.xlabel('k'); plt.ylabel('inertia')
```

34.4 Silhouette score — another way to evaluate clustering

Silhouette score measures how similar each point is to its own cluster vs other clusters. For a single point: $s = (b - a) / \max(a, b)$, where a = mean distance to points in same cluster, b = mean distance to points in nearest other cluster.

- **+1**: point is far from neighboring clusters (well-clustered).
- **0**: point is on the boundary.
- **-1**: point is in the wrong cluster.

```
from sklearn.metrics import silhouette_score
score = silhouette_score(X, km.labels_) # higher = better
```

34.5 K-Means properties

- **Needs k upfront.** You must choose the number of clusters in advance.
- **Sensitive to feature scaling.** Always scale before K-Means.
- **Sensitive to outliers.** One extreme point can pull a centroid far away.
- **Assumes spherical, equal-sized clusters.** Struggles with elongated, varying-density, or non-convex clusters.
- **Output depends on initialization.** Always use k-means++ and n_init > 1.
- **Fast.** Linear in number of points and dimensions per iteration.

NOTE: Common PYQ: 'Which of these algorithms is impacted by feature scaling?' K-Means, KNN, SVM, and PCA are the classic four (because all use distance/variance). Trees, Random Forest, and Naive Bayes are NOT impacted.

35. Hierarchical Agglomerative Clustering (HAC)

HAC is a different family of clustering. Unlike K-Means, it doesn't require you to specify k upfront and produces a **dendrogram** — a tree showing how clusters merge at different distances. You can cut the dendrogram at any height to get any number of clusters.

35.1 The algorithm (bottom-up)

- 1. Start with every point as its own cluster (n clusters total).
- 2. Find the two **CLOSEST** clusters and merge them (now $n-1$ clusters).
- 3. Repeat until you have one big cluster.
- 4. Cut the resulting tree at any level to get the number of clusters you want.

There's also a **top-down (divisive)** variant that starts with all points in one cluster and recursively splits. Less common in practice; sklearn implements only bottom-up (agglomerative).

35.2 Linkage criteria — how to measure distance between clusters

When the algorithm picks the "two closest clusters," it needs a way to measure distance between two CLUSTERS (each containing many points). Several options:

Linkage	How distance is computed	Behavior
'single'	Distance between the two CLOSEST points across clusters	Can produce long, chain-like clusters; sensitive to outliers
'complete'	Distance between the two FARTHEST points across clusters	Produces compact, round clusters; outlier-sensitive
'average'	Mean distance between all pairs	Compromise between single and complete
'ward' (default)	Increase in within-cluster variance after merging	Produces compact clusters; only works with Euclidean distance

35.3 Distance metrics

Common distance/similarity metrics for HAC:

- **Euclidean**: straight-line distance. Default. Required for Ward linkage.
- **Manhattan**: sum of absolute differences.
- **Cosine**: angle between vectors. Good for high-dim and text.
- **Jaccard**: for binary/set data; intersection over union.

35.4 AgglomerativeClustering in sklearn

```
from sklearn.cluster import AgglomerativeClustering

clustering = AgglomerativeClustering(
    n_clusters=3,          # how many clusters you want
    metric='euclidean',    # 'euclidean', 'manhattan', 'cosine', etc.
    linkage='ward'         # 'ward', 'complete', 'average', 'single'
)
clustering.fit(X)
clustering.labels_        # cluster assignments
```

NOTE: Important: HAC has no predict() method. Once you fit, you have labels for THOSE specific points. To assign new points, you'd need to refit with the new data added.

35.5 K-Means vs HAC — the comparison PYQs ask

	K-Means	HAC
Need k upfront?	Yes	No (cut dendrogram anywhere)
Approach	Centroid-based, iterative	Bottom-up merging
Output	Cluster labels + centroids	Cluster labels + dendrogram
Assumes cluster shape?	Spherical	Depends on linkage
Outlier-sensitive?	Very	Depends on linkage (single most sensitive)
Speed on big data	Fast ($O(nk)$ per iter)	Slow ($O(n^2)$ or $O(n^3)$)
Best for	Big data, you know k	Smaller data, exploring cluster structure

WATCH OUT: Two PYQ traps from 2025 papers:

1. "HAC starts with all points in a single cluster and recursively splits." FALSE for agglomerative (bottom-up). That's the divisive variant.
2. "HAC requires the number of clusters before training." FALSE. You can cut the dendrogram at any level afterwards. (Note: sklearn's implementation does require `n_clusters` at instantiation, but the algorithm itself doesn't need it.)

36. End-to-End ML Project Workflow

Your syllabus has a series called "Looking at the big picture" through "Fine-tuning ML models." These cover the standard workflow for a real ML project. Here's the canonical sequence:

Step	What to do	Tools / sklearn classes
1. Frame the problem	Is it classification or regression? What's the metric? What does success look like?	Pen and paper
2. Get the data	Load it. Understand its shape and structure.	load_*, fetch_*, pd.read_csv
3. Explore (EDA)	Histograms, scatter plots, correlations. Look at distributions, check for NaN.	df.describe(), df.hist(), df.corr()
4. Train/test split	Hold out a test set IMMEDIATELY. Don't peek at it.	train_test_split
5. Prepare the data	Impute, encode, scale, feature engineer. Wire into a Pipeline.	SimpleImputer, OneHotEncoder, StandardScaler, ColumnTransformer, Pipeline
6. Try several models	Linear regression, decision tree, random forest, etc. Use cross-validation.	cross_val_score, multiple model classes
7. Fine-tune	Hyperparameter search the most promising models.	GridSearchCV, RandomizedSearchCV
8. Evaluate on test set	ONCE. With the final tuned model.	model.score, classification_report
9. Present & deploy	Communicate results, save the model.	joblib.dump, pickle

WATCH OUT: The cardinal rule: NEVER touch your test set during training, exploration, or tuning. Only use it ONCE, at the end, to estimate final performance. Otherwise you leak information and your test score becomes meaningless.

36.1 Stratified splitting for classification

When classes are imbalanced, a random train/test split might give you a test set with very few samples of the minority class. **Stratified splitting** preserves class proportions in both sets:

```
# For train/test split:
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)

# For cross-validation:
from sklearn.model_selection import StratifiedKFold
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```

37. Cross-Validation

A single train/test split gives you ONE estimate of test performance, which can vary a lot based on which samples landed in test. Cross-validation does it many times and averages, giving a more stable estimate. It's also essential for hyperparameter tuning.

37.1 K-Fold cross-validation

Split the training data into K equal folds. For each fold, train on the other K-1 folds and validate on this fold. You get K scores; report their mean (and std).

- **Common choices:** K=5 or K=10.
- **Pros:** uses all data for both training and validation.
- **Cons:** K times slower than a single train/val split.

```
from sklearn.model_selection import KFold, cross_val_score

kf = KFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(model, X_train, y_train, cv=kf, scoring='accuracy')
scores.mean(), scores.std()
```

37.2 StratifiedKFold — for classification

Like KFold, but each fold preserves the class distribution of the full dataset. **Always use this instead of plain KFold for classification** — especially with imbalanced classes.

```
from sklearn.model_selection import StratifiedKFold
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
cross_val_score(model, X, y, cv=skf, scoring='f1')
```

NOTE: Sklearn shortcut: When you pass **cv=5** as an integer to `cross_val_score` or `GridSearchCV` with a CLASSIFIER, sklearn automatically uses `StratifiedKFold`. With a REGRESSOR, it uses plain `KFold`. So **cv=5** is usually safe.

37.3 Other CV strategies

Strategy	How it works	Use when
LeaveOneOut (LOO)	n folds: each fold has 1 sample	Very small datasets; very expensive
LeavePOut	Each fold leaves p samples out	Even more expensive, rarely used
ShuffleSplit	Random train/test splits, repeated n_splits times	When you want more flexibility than KFold
StratifiedShuffleSplit	ShuffleSplit + class-balanced	Imbalanced classes with custom split sizes
TimeSeriesSplit	Train on past, validate on future, in expanding windows	Time series — never shuffle in time!
GroupKFold	Ensures groups (e.g., users) don't appear in both train and val	Data has natural grouping (per-user, per-session)

37.4 The cross_val_score / cross_validate functions

```
from sklearn.model_selection import cross_val_score, cross_validate

# Single metric:
scores = cross_val_score(model, X, y, cv=5, scoring='accuracy')

# Multiple metrics:
results = cross_validate(model, X, y, cv=5,
                        scoring=['accuracy', 'precision', 'recall', 'f1'],
                        return_train_score=True)
results['test_accuracy'].mean()
results['train_accuracy'].mean() # to spot overfitting
```

TIP: Spotting overfitting with CV: if train_score is much higher than test_score, you're overfitting. If both are low, you're underfitting. Set **return_train_score=True** in cross_validate to see both.

37.5 The scoring parameter

Pass a string to **scoring=** to pick a metric. Some common options:

Task	Common scoring strings
Classification	'accuracy', 'precision', 'recall', 'f1', 'roc_auc'
Multi-class classification	'f1_macro', 'f1_micro', 'f1_weighted'
Regression	'r2', 'neg_mean_squared_error', 'neg_mean_absolute_error', 'neg_root_mean_squared_error'

WATCH OUT: The 'neg_' prefix: Sklearn's convention is that **higher is better** for scoring. But MSE and MAE are LOSSES (lower is better). So sklearn negates them, giving you 'neg_mean_squared_error' — the higher (less negative) it is, the better. When you read these results, remember they're negative numbers.

38. GridSearchCV & RandomizedSearchCV

Hyperparameter tuning combined with cross-validation. These find the best parameter values for your model by trying many combinations and using CV to score each one. Almost guaranteed to appear in your exam.

38.1 GridSearchCV — exhaustive search

Try every combination of every value you specify.

```
from sklearn.model_selection import GridSearchCV
from sklearn.svm import SVC

param_grid = {
    'C': [0.1, 1, 10, 100],
    'kernel': ['linear', 'rbf'],
    'gamma': ['scale', 'auto']
}

grid = GridSearchCV(
    SVC(),
    param_grid,
    cv=5,                                # 5-fold cross-validation
    scoring='accuracy',
    n_jobs=-1,                           # parallel
    refit=True,                           # refit best on full training data (default)
    return_train_score=False
)
grid.fit(X_train, y_train)

grid.best_params_      # the winning combination
grid.best_score_       # mean CV score of the winning combination
grid.best_estimator_   # the actual fitted model with best params
grid.cv_results_       # full results dict for all combinations
```

38.2 Counting combinations — the math PYQs love

Total combinations = product of the lengths of all lists in param_grid. Total fits = combinations × cv folds. Plus 1 if refit=True (default), since GridSearchCV refits the best on all training data after CV.

NOTE: Worked example: with the param_grid above, we have 4 C values × 2 kernels × 2 gammas = **16 combinations**. With cv=5, that's **80 model fits** during CV, plus 1 final refit = 81 total fits. Most PYQs ask for the combinations (16) or the total CV fits (80).

38.3 GridSearchCV with a Pipeline

When tuning a Pipeline, use the **step_name__param** notation:

```
from sklearn.pipeline import Pipeline

pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('model', LogisticRegression())
])

param_grid = {
```

```

        'model__C': [0.01, 0.1, 1, 10],
        'model__penalty': ['l1', 'l2'],
        'scaler__with_mean': [True, False]
    }

    grid = GridSearchCV(pipe, param_grid, cv=5)
    grid.fit(X_train, y_train)

```

38.4 RandomizedSearchCV — random sampling

When the grid is too large, RandomizedSearchCV samples **n_iter** combinations randomly. Often finds nearly-optimal parameters in much less time.

```

from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import uniform, randint

param_dist = {
    'C': uniform(0.001, 100),          # continuous: random float in [0.001, 100.001]
    'gamma': uniform(0.001, 1),
    'kernel': ['rbf', 'linear']        # discrete: random pick
}

search = RandomizedSearchCV(
    SVC(),
    param_dist,
    n_iter=50,                          # try 50 random combinations
    cv=5,
    random_state=42,
    n_jobs=-1
)
search.fit(X_train, y_train)

```

38.5 GridSearchCV vs RandomizedSearchCV

	GridSearchCV	RandomizedSearchCV
Search strategy	Every combination	n_iter random combinations
Total fits	Combinations × folds	n_iter × folds
Parameter spaces	Discrete lists only	Lists OR distributions (continuous)
When to use	Few hyperparameters, small grid	Many hyperparameters, big grid
Reproducibility	Always same result	Set random_state for consistent picks
Cost	Can explode combinatorially	Fixed budget (n_iter)

NOTE: Other ways to tune (less common): HalvingGridSearchCV and HalvingRandomSearchCV use successive halving (start with many configs and few resources, halve configs and double resources each round). Faster than full grid for big searches.

39. Transfer Learning

Transfer learning is in your syllabus (Week 12). The core idea: take a model that's already been trained on a HUGE dataset (e.g., ImageNet for images, or BERT for text), and adapt it to YOUR specific (usually smaller) task. You inherit all the patterns it learned without training from scratch.

39.1 Why transfer learning works

Deep neural networks learn in hierarchies. Early layers learn very general features (edges, textures, colors for images; basic word patterns for text). Later layers learn more task-specific features. The general features are useful for almost any related task, so we can reuse them.

39.2 Two main approaches

Feature extraction

Freeze the pre-trained model's weights and use it as a fixed feature extractor. Add a new small classifier on top (just an output layer or a couple dense layers) and train **ONLY** those new layers.

- **Pros:** very fast, needs little data.
- **Cons:** the pre-trained features may not be perfectly suited to your task.

Fine-tuning

Start with the pre-trained weights but **unfreeze** some (or all) layers and continue training with a **SMALL** learning rate. The model adapts to your data but doesn't lose what it already knew.

- **Pros:** often higher accuracy than feature extraction.
- **Cons:** needs more data and compute. Risk of overwriting useful features ("catastrophic forgetting").

39.3 Typical transfer learning workflow

- 1. Load a pre-trained model (e.g., ResNet, VGG, BERT) from a model hub.
- 2. Remove the original output layer (which was for the original task).
- 3. Add new output layer(s) sized for your task.
- 4. Decide which layers to freeze:
 - • Small dataset, similar to original: freeze most, train new top.
 - • Small dataset, different from original: freeze early layers, retrain later.
 - • Large dataset, similar: fine-tune the whole thing with small lr.
 - • Large dataset, different: fine-tune everything; or just train from scratch.
- 5. Train with a **SMALL** learning rate (1e-4 or smaller for fine-tuning).

NOTE: Sklearn doesn't really do transfer learning; it's a deep-learning concept. Your course likely uses TensorFlow/Keras or PyTorch for the practical demos. For exam purposes, focus on understanding the concept: pre-trained model + freeze/fine-tune + new task head.

39.4 Common pre-trained models

Domain	Famous pre-trained models	Original training task
Computer Vision	ResNet, VGG, Inception, EfficientNet	ImageNet (1000-class image classification)
Natural Language	BERT, GPT, T5, RoBERTa	Masked language modeling on huge text corpora
Audio	Wav2Vec, HuBERT	Self-supervised on audio

40. Bias-Variance Tradeoff & Overfitting

Foundational concept. Every ML model has two sources of error beyond irreducible noise:

40.1 The two errors

Bias: the error from incorrect assumptions or oversimplified models. A linear model trying to fit a curved relationship has high bias — it CAN'T capture the true pattern. **High bias = underfitting.**

Variance: the error from being too sensitive to specific training data. A super complex model that memorizes every training point will give wildly different predictions for slightly different training sets. **High variance = overfitting.**

40.2 The tradeoff

There's an inherent tension. Reducing bias usually increases variance and vice versa. The art of ML is finding the sweet spot.

Symptom	Likely cause	Fix
Train score low, test score low (both bad)	High bias underfitting	/ Use more complex model, more features, less regularization
Train score high, test score much lower	High variance overfitting	/ Simpler model, more data, regularization, ensembles
Train and test scores both high & close	Just right	Don't change anything

40.3 Specific fixes for overfitting

- **Get more training data** (most effective when feasible).
- **Reduce model complexity:** shallower trees, smaller MLP, lower-degree polynomials.
- **Add regularization:** Ridge/Lasso for linear, max_depth/min_samples_leaf for trees, alpha for MLP.
- **Use ensembles:** Random Forest reduces variance via averaging.
- **Cross-validation:** use it to detect overfitting and tune hyperparameters.
- **Drop irrelevant features** via feature selection.

40.4 Bias and variance in common algorithms

Algorithm	Tends to have	When tuned poorly
Linear/Logistic Regression (no regularization)	Low variance, can have high bias	Underfits curved data
Decision Tree (deep)	Low bias, high variance	Memorizes training set
KNN with $k=1$	Low bias, very high variance	Noisy
KNN with $k=N$	High bias, low variance	Predicts the global majority
Random Forest	Low bias, low variance (the magic!)	Generally robust
Naive Bayes	Higher bias from independence assumption	Misses feature interactions

41. The Final Cheat Sheet

One-page reference for the day before your exam. Memorize the key facts; use the table to look up specifics.

41.1 Critical facts to memorize

- **Bunch object:** `load_*/fetch_*` both return Bunch with `.data`, `.target`, `.feature_names`, `.target_names`.
- **load_* vs fetch_* vs make_*:** small bundled / large downloaded / synthetic generated.
- **fit_transform on train, transform on test.** Never fit_transform on test.
- **df.iloc** end-exclusive (Python rule); **df.loc** end-INCLUSIVE.
- **StandardScaler:** mean=0, std=1. **MinMaxScaler:** range [0, 1]. **RobustScaler:** outlier-proof. **MaxAbsScaler:** [-1, 1], preserves sparsity.
- **Don't need scaling:** Decision Tree, Random Forest, Naive Bayes.
- **Need scaling:** KNN, K-Means, SVM, PCA, Logistic Regression with regularization, MLP, SGD.
- **OrdinalEncoder:** ordered features. **OneHotEncoder:** nominal features. **LabelEncoder:** target only.
- **SimpleImputer strategies:** mean, median, most_frequent, constant. KNNImputer for smarter.
- **chi2 needs non-negative features.** Don't use after StandardScaler.
- **PCA always scale first.** `n_components` can be int, float (variance ratio), or 'mle'.
- **Pipeline:** sequential. **ColumnTransformer:** parallel on different cols. **FeatureUnion:** parallel on same data.
- **Pipeline params:** use `step_name__param` notation in GridSearchCV.
- **R² range:** -∞ to 1.0. $R^2 = 1 - SS_{res}/SS_{tot}$.
- **Ridge (L2):** shrinks coefficients, doesn't zero. **Lasso (L1):** zeros some out (feature selection).
- **LogisticRegression: C is INVERSE of regularization.** Smaller C = stronger reg.
- **Confusion matrix (sklearn):** rows=actual, cols=predicted. `cm[1,1]`=TP. `cm[0,0]`=TN.
- **Precision = TP/(TP+FP). Recall = TP/(TP+FN). F1 = 2PR/(P+R).**
- **OvR:** n classifiers. **OvO:** n(n-1)/2 classifiers. SVM uses OvO.
- **KNN bottleneck is PREDICTION, not training.**
- **Naive Bayes:** GaussianNB (continuous), MultinomialNB (counts), BernoulliNB (binary).
- **SVM kernels:** linear, poly, rbf (default), sigmoid. C = inverse regularization.
- **Decision Tree:** Gini = $1 - \sum p_i^2$. Entropy = $-\sum p_i \log_2 p_i$.
- **Bagging reduces variance. Boosting reduces bias.**
- **RandomForest:** `n_estimators` = number of trees. `max_features='sqrt'` is default.
- **Soft voting averages probabilities;** hard voting picks majority class.
- **K-Means inertia:** sum of squared distances from points to their centroid. Lower = tighter.
- **HAC linkages:** single (closest pair), complete (farthest pair), average, ward (default, Euclidean only).
- **StratifiedKFold** for classification (preserves class ratios). KFold for regression.
- **cv=5 with classifier** → sklearn auto-uses StratifiedKFold.
- **GridSearchCV combinations:** multiply lengths of all param lists. Total fits = combos × cv (+1 for refit).

- **'neg_' scoring**: sklearn negates losses so higher = better. e.g. 'neg_mean_squared_error'.
- **warm_start=True**: continue training from previous weights (SGDRegressor, MLP, Perceptron).
- **random_state**: fix for reproducibility.

41. Cheat Sheet (continued)

41.2 Algorithm choice quick guide

You want...	Reach for...
Simple, interpretable regression	LinearRegression
Regression with many features, want feature selection	Lasso
Regression with correlated features	Ridge
Regression on big data	SGDRegressor
Probabilistic binary classifier, fast	LogisticRegression
Quick non-parametric classifier	KNeighborsClassifier
Text classification (counts/TF-IDF)	MultinomialNB
Best general-purpose classifier without tuning	RandomForestClassifier
Best with tuning & complex data	GradientBoostingClassifier or XGBoost
Small data with non-linear boundary	SVC with RBF kernel
Highly interpretable rules	DecisionTreeClassifier
Tabular data, small/medium	RandomForest or GradientBoosting
Tabular data, complex non-linear	MLPClassifier
Cluster grouping when you know k	KMeans
Cluster exploration without knowing k	AgglomerativeClustering
Reduce dimensions for visualization	PCA(n_components=2)
Preprocess heterogeneous columns	ColumnTransformer

41.3 Default scoring metric per sklearn class

When you call `model.score(X, y)`:

- **Classifiers** (LogisticRegression, KNeighborsClassifier, DecisionTreeClassifier, RandomForestClassifier, SVC, MLPClassifier...): return **accuracy**.
- **Regressors** (LinearRegression, Ridge, Lasso, KNeighborsRegressor, RandomForestRegressor, MLPRegressor...): return **R² score**.
- **KMeans**: returns **negative inertia** (so higher = better, sklearn convention).

41.4 Top 10 PYQ traps

1. **fit_transform on test data** (data leakage). Always transform only.
2. **df.loc end-INCLUSIVE** vs df.iloc end-EXCLUSIVE.
3. **LogisticRegression C** is INVERSE of regularization (opposite of Ridge alpha).
4. **KNN bottleneck is prediction**, not training.
5. **Trees and Naive Bayes don't need scaling**. Everything else does (mostly).
6. **HAC bottom-up** doesn't need k upfront; divisive top-down does (algorithmically).

- 7. **warm_start=True** is the right answer for "continue training from previous weights."
- 8. **random_state** is needed for reproducibility — without it, results vary.
- 9. **Soft voting averages probabilities**; doesn't pick max-sum-of-probs.
- 10. **OneHotEncoder** for nominal, **OrdinalEncoder** for ordered, **LabelEncoder** for target.

42. Last-Day Revision Strategy

42.1 Day before exam

- **Morning (3 hours):** Take 2025 Dec 18 paper under exam conditions (timer on, no looking up). Be brutal with yourself.
- **Afternoon (2 hours):** Review the paper. For every question you got wrong, look up the relevant sklearn class on the official docs (<https://scikit-learn.org/>) and skim its parameter list.
- **Evening (1 hour):** Read this Part 4 cheat sheet TWICE. Specifically, memorize the bullet list in section 41.1.
- **Night:** SLEEP. Don't pull an all-nighter. Tired brains miss obvious tricks.

42.2 Morning of exam

- Re-read sections 41.1 & 41.4 (the cheat sheet bullet list and Top 10 PYQ traps) once over coffee.
- Don't try to learn anything new. If you don't know it now, you won't learn it in 30 minutes.
- Eat breakfast. Hydrate.

42.3 During the exam

- **Read every question twice.** The IIT-M exam loves traps where one word changes everything (e.g., "recall" vs "precision"; "axis=0" vs "axis=1"; "smaller C" vs "larger C").
- **For code-output questions,** trace through line by line on rough paper. Don't trust mental math.
- **For confusion matrix questions,** always restate: TN, FP, FN, TP first. Then compute precision, recall, etc.
- **For GridSearchCV combination counts,** just multiply the lengths of the param lists. Add cv folds if asked for total fits.
- **If stuck,** mark it and move on. Come back at the end.
- **For multi-select (MSQ),** read EVERY option. Often 2-3 are correct, and missing one loses partial credit.
- **For numeric (SA) questions,** read the precision requested ("to 2 decimals").
- **Before submitting,** review every flagged question once more.

42.4 Mental model for the exam

Your IIT-M MLP exam is about **recognizing what each sklearn snippet does and predicting behavior**. It is NOT about deriving math from scratch or coming up with creative ML approaches. Most questions can be answered by:

- Knowing what class is being used (the import statement is your friend).
- Knowing what its key parameters do.
- Knowing the rough output shape and properties.

If you've absorbed Parts 1-4, you have all of this knowledge. Trust it.

42.5 What to do AFTER the exam

Whatever happens, you've now learned a substantial amount of practical machine learning — far more than most students who actually attended their lectures. The skills you've built (sklearn fluency, knowing the canonical workflow, understanding the bias-variance tradeoff) are useful for projects, interviews, and any future ML work. The exam is just a checkpoint.

Final Word

You went from "haven't studied anything" to "have a coherent grasp of the entire MLP syllabus" in 7 days. That's a real achievement. Whatever score you get, you'll walk out understanding more than you did walking in — and that's what learning is for.

Good luck. Go ace it.

Where to find what

Topic	Part
Pandas, Series/DataFrame, loc/iloc	Part 1
sklearn API, Bunch, load_/fetch_/make_	Part 1
Imputation, Encoding, Scaling, Outliers	Part 1
Feature Selection, PCA	Part 2
Pipeline, ColumnTransformer, FeatureUnion	Part 2
Linear, Polynomial, Ridge, Lasso, ElasticNet	Part 2
SGDRegressor, Regression Metrics	Part 2
Logistic Regression, OvR/OvO	Part 3
Confusion Matrix, Precision, Recall, F1, ROC-AUC	Part 3
KNN, Naive Bayes, SVM, Decision Trees	Part 3
Voting, Bagging, Random Forest, Boosting	Part 3
MLP / Perceptron	Part 3
K-Means, HAC	Part 4
Cross-validation strategies	Part 4
GridSearchCV, RandomizedSearchCV	Part 4
Transfer Learning	Part 4
Bias-Variance, Cheat Sheet, Exam Strategy	Part 4

MLP Exam Prep

Concept Mastery Guide — Part 5 (Gap-closer)

Recommender Systems & Time Series

These two topics appeared in the Dec 2025 paper (Q1-Q4 and Q11) and are confirmed in your syllabus (Weeks 10 and 11). They're worth roughly **10-15 marks** based on recent papers. The good news: both are conceptually self-contained and you don't need deep math to score on them. Read this part once carefully and you've patched the gap.

What's in Part 5

#	Section	Why it matters
43	Recommender Systems — the big picture	Conceptual foundation
44	Content-Based Filtering	Dec 2025 Q3 (cold start)
45	Collaborative Filtering (user-user & item-item)	Frequently asked
46	Cosine Similarity (computation)	Dec 2025 Q11 (numeric)
47	Matrix Factorization & SVD	Dec 2025 Q4
48	Time Series — pandas basics	Foundation for time series Qs
49	Time Resampling	Common in Week 11
50	Trend, Seasonality & Decomposition (Classical & STL)	Conceptual
51	Stationarity & ADF Test	Dec 2025 Q1, Q2
52	Making data stationary (differencing)	Practical follow-up
53	Self-check questions & final wrap-up	Test yourself

43. Recommender Systems — The Big Picture

A recommender system predicts what items a user will like. Netflix recommending movies, Amazon suggesting products, YouTube queuing up videos — all the same problem. The input is usually a sparse **user-item interaction matrix** (each cell = a rating, or 1/0 for clicked/not-clicked); the output is a ranked list of items for each user.

43.1 The user-item matrix

Imagine a table with **users as rows** and **items (movies, products) as columns**. Each cell holds a rating (e.g., 1-5 stars) if the user has rated that item, or NaN if they haven't. The matrix is overwhelmingly empty — most users have rated only a tiny fraction of items. The recommender's job is to fill in the blanks.

	Movie A	Movie B	Movie C	Movie D
User 1	5	?	3	?
User 2	?	4	?	2
User 3	4	5	?	?
User 4	?	?	5	1

43.2 Three main approaches

Approach	Uses what?	Strength	Weakness
Content-Based	Item features (genre, cast)	Works for new items	Needs item metadata; narrow recommendations
Collaborative Filtering	User-item interactions only	No metadata needed; finds surprising connections	Cold start problem
Hybrid	Both	Best of both	More complex

43.3 The cold start problem

Cold start happens when you don't have enough interaction data to make recommendations:

- **New user cold start:** a brand-new user has no ratings yet. Collaborative filtering can't help — there's nothing to compare them with.
- **New item cold start:** a brand-new item has no ratings yet. Collaborative filtering can't help — nobody has interacted with it. But content-based CAN help, because the item's genre/director/etc. metadata is available immediately.

NOTE: Dec 2025 PYQ Q3: 'A newly launched movie has no ratings, but metadata such as genre, cast, and director is available. Which algorithm can still recommend it?' Answer: **Content-based filtering**. Collaborative filtering (both user-based and item-based) needs interaction data, which doesn't exist for a brand-new item. Matrix factorization without regularization is also collaborative — same problem.

44. Content-Based Filtering

Recommend items **similar to those the user already liked**, where similarity is measured using item features (not user-user similarity). If you liked "Inception" (sci-fi, Christopher Nolan, 2010), recommend other sci-fi Christopher Nolan films.

44.1 The procedure

- 1. Build a **feature vector** for each item. Features can be one-hot encoded genre, TF-IDF of plot description, director, cast, runtime, etc.
- 2. Build a **user profile vector** by aggregating (e.g., averaging) the feature vectors of items the user liked.
- 3. For a new candidate item, compute **similarity** between the candidate's feature vector and the user profile.
- 4. Recommend items with the highest similarity scores.

44.2 Example

Suppose 3 movies have these one-hot encoded genre vectors:

Movie	Action	Comedy	Romance	Sci-Fi
Movie A	1	0	0	1
Movie B	0	1	1	0
Movie C	1	0	0	1

User 1 has rated Movie A highly. To find similar items, compute the similarity between Movie A's vector [1, 0, 0, 1] and Movie B [0, 1, 1, 0] vs Movie C [1, 0, 0, 1]. Movie C is identical to A — perfect similarity. Recommend Movie C.

44.3 Strengths and weaknesses

Strengths	Weaknesses
No "new item" cold start — works as soon as item is added	Needs rich item metadata
Recommendations are explainable ("because you liked X")	Recommends only items similar to past taste — no surprises
No user data needed beyond their own history	Doesn't capture quality (just genre matching)
Easy to implement	Limited by quality of features

TIP: Content-based is the go-to answer for any PYQ that mentions: "new item just launched, no ratings yet, but has metadata." That phrasing locks in the answer.

45. Collaborative Filtering

Collaborative filtering ignores item metadata entirely. It works on the principle: "users who agreed in the past will agree in the future." The only data it uses is the user-item interaction matrix.

45.1 User-User Collaborative Filtering

"Find users similar to me; recommend what THEY liked."

- 1. Compute similarity between users (using their rating vectors).
- 2. For each unrated item, predict your rating as a weighted average of similar users' ratings.
- 3. Recommend the top-predicted items.

Example: if User 1 and User 3 both rated Movies A and B similarly, they're similar users. If User 3 loved Movie D (which User 1 hasn't seen), recommend Movie D to User 1.

Limitations of user-user CF

- **Doesn't scale well:** with millions of users, computing pairwise user similarities is expensive.
- **Sparse data:** most pairs of users have very few co-rated items, making similarities unstable.
- **User tastes change over time,** so similarities drift.

45.2 Item-Item Collaborative Filtering

"Find items similar to ones I've liked; recommend those."

- 1. Compute similarity between items (using their column vectors of ratings).
- 2. For each unrated item, predict the user's rating from similar items they HAVE rated.
- 3. Recommend the top-predicted items.

Example: if Movie A and Movie C are both rated highly by the same set of users, they're similar items. If you rated Movie A 5 stars and haven't seen Movie C, predict you'll rate C highly too.

Why item-item is preferred in practice

- **Items change less often than users** (new movies trickle in; new users sign up daily). Item similarities are more stable.
- **Items are usually fewer than users** (Amazon has billions of users but millions of products), so the similarity matrix is smaller.
- **Recommendations are interpretable:** "Customers who bought X also bought Y."

45.3 User-User vs Item-Item — comparison

	User-User CF	Item-Item CF
What's compared	Users (rows)	Items (columns)
Recommendation reason	"People like you liked..."	"You liked X, here's similar Y"
Stability over time	Less stable (users change)	More stable (items don't change much)
Scalability	Poor when many users	Better when items << users
Used by	Older systems	Amazon, Netflix (modern)

WATCH OUT: Both forms of CF suffer from the cold start problem. A new user has no ratings, so we can't find similar users for them. A new item has no ratings, so we can't find similar items. **Content-based filtering doesn't have this issue for new items.**

46. Cosine Similarity (Computation)

All these recommender approaches need a way to measure "similarity" between two vectors. The most common choice is **cosine similarity**, which measures the angle between two vectors. The Dec 2025 PYQ Q11 asks you to compute cosine similarity by hand — you must know this formula.

46.1 The formula

$$\text{cosine_similarity}(\mathbf{a}, \mathbf{b}) = (\mathbf{a} \cdot \mathbf{b}) / (\|\mathbf{a}\| \cdot \|\mathbf{b}\|)$$

Breaking this down:

- $\mathbf{a} \cdot \mathbf{b}$ = dot product = $a_1b_1 + a_2b_2 + \dots + a_nb_n$
- $\|\mathbf{a}\|$ = magnitude (Euclidean norm) of $\mathbf{a} = \sqrt{a_1^2 + a_2^2 + \dots + a_n^2}$
- $\|\mathbf{b}\|$ = same for $\mathbf{b}$

46.2 Range and meaning

- **+1**: vectors point in the same direction (perfectly similar).
- **0**: vectors are orthogonal (no similarity).
- **-1**: vectors point in opposite directions (perfectly dissimilar).

For non-negative vectors (like rating vectors or one-hot features), cosine similarity is always between 0 and 1.

46.3 Worked example (the kind of question you'll see)

Item A has feature vector $\mathbf{a} = [3, 4, 0]$. Item B has $\mathbf{b} = [4, 0, 3]$. What's the cosine similarity?

Step 1 — Dot product:

$$\mathbf{a} \cdot \mathbf{b} = (3)(4) + (4)(0) + (0)(3) = 12 + 0 + 0 = \mathbf{12}$$

Step 2 — Magnitudes:

$$\|\mathbf{a}\| = \sqrt{3^2 + 4^2 + 0^2} = \sqrt{9 + 16 + 0} = \sqrt{25} = \mathbf{5}$$

$$\|\mathbf{b}\| = \sqrt{4^2 + 0^2 + 3^2} = \sqrt{16 + 0 + 9} = \sqrt{25} = \mathbf{5}$$

Step 3 — Divide:

$$\text{cosine_similarity} = 12 / (5 \times 5) = 12 / 25 = \mathbf{0.48}$$

46.4 In sklearn

```
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

a = np.array([[3, 4, 0]]) # 2D shape required
b = np.array([[4, 0, 3]])

sim = cosine_similarity(a, b)
# array([[0.48]])

# Compute pairwise similarity for many vectors at once:
items = np.array([[3, 4, 0], [4, 0, 3], [1, 1, 1]])
sim_matrix = cosine_similarity(items)
# 3x3 matrix; sim_matrix[i, j] = similarity between item i and item j
```

46.5 Other similarity / distance metrics

Metric	Formula	Use when
Cosine similarity	$(a \cdot b) / (a b)$	Direction matters more than magnitude (default for recommenders)
Euclidean distance	$\sqrt{\sum (a_i - b_i)^2}$	Geometric closeness in feature space
Manhattan distance	$\sum a_i - b_i $	Grid-like, robust to outliers
Pearson correlation	Covariance / (std_a × std_b)	Measures linear relationship; like cosine on mean-centered vectors
Jaccard similarity	$ A \cap B / A \cup B $	Set comparison (binary features only)

NOTE: Cosine vs Euclidean for recommenders: Cosine is preferred because it ignores magnitude. Two users with rating vectors [5, 4, 3] and [3, 2, 1] have similar TASTE (same ranking pattern) even though one is a generally generous rater. Euclidean distance would say they're dissimilar; cosine says they're identical.

47. Matrix Factorization & SVD

Matrix factorization is a more powerful collaborative filtering technique. The idea: decompose the giant sparse user-item matrix **R** into the product of two smaller **dense** matrices — one for users (**U**) and one for items (**V**).

47.1 The decomposition

$$\mathbf{R} \approx \mathbf{U} \times \mathbf{V}^T$$

Where **R** is the **m users × n items** rating matrix, **U** is **m users × k latent factors**, and **V** is **n items × k latent factors**. The number **k** is much smaller than **m** or **n** — typical **k** = 10 to 200.

47.2 What are "latent factors"?

Latent factors are **hidden dimensions of taste** that the algorithm discovers automatically. For movies, they might roughly correspond to "action-vs-romance," "serious-vs-comedy," "old-vs-new" — but you don't tell the algorithm what they are. Each user has a position in this **k**-dimensional taste space (a row of **U**); each item has a position in the same space (a row of **V**).

To predict User *i*'s rating for Item *j*: $\mathbf{R}_{ij} \approx \mathbf{U}_i \cdot \mathbf{V}_j$ — the dot product of their latent vectors. If they line up well in latent space, predicted rating is high.

47.3 SVD — Singular Value Decomposition

SVD is a classical linear algebra technique that decomposes any matrix as:

$$\mathbf{R} = \mathbf{U} \times \Sigma \times \mathbf{V}^T$$

Where Σ is a diagonal matrix of **singular values** sorted from largest to smallest. By keeping only the top **k** singular values (and the corresponding columns of **U** and **V**), you get a rank-**k** approximation of **R** that captures the dominant structure.

For recommender systems, SVD is used to **reduce dimensionality of the user-item matrix**, filling in missing values implicitly. This is exactly what's asked in the Dec 2025 PYQ Q4.

NOTE: Dec 2025 Q4 PYQ: 'Performing SVD on a user-item matrix aims to:' Answer: **Reduce dimensionality**. Distractors: "Increase sparsity" (no, opposite), "Delete user history" (no), "Improve data imbalance" (no).

47.4 The big picture — matrix factorization workflow

1. Start with the sparse user-item rating matrix **R**.
2. Initialize **U** and **V** randomly.
3. Use gradient descent to minimize the prediction error on KNOWN ratings: minimize $\sum (\mathbf{R}_{ij} - \mathbf{U}_i \cdot \mathbf{V}_j)^2$ over (i,j) where $\mathbf{R}_{ij}$ is observed.
4. Add regularization (L2 on **U** and **V**) to prevent overfitting.
5. After training, predict ratings for unrated cells: predicted $\mathbf{R}_{ij} = \mathbf{U}_i \cdot \mathbf{V}_j$.
6. Recommend items with the highest predicted ratings.

47.5 Strengths of matrix factorization

- **Captures latent structure** that simple user-user / item-item similarity misses.
- **Handles sparsity**: works even when most cells are missing.
- **Compresses the data**: an $m \times n$ matrix is replaced by $m \times k + n \times k$ parameters, much smaller.
- **Famous winner** of the 2009 Netflix Prize.
- **Still suffers from cold start** — new users/items have no learned latent vector.

48. Time Series — Pandas Basics

A **time series** is data indexed in time order: temperature each hour, stock price each minute, monthly sales totals. The crucial constraint: **you cannot shuffle the data** — the order matters because each value depends on what came before it.

48.1 Pandas datetime index

The cornerstone of time-series in pandas is the **DatetimeIndex**. Convert any string column to actual datetime objects with **pd.to_datetime**:

```
import pandas as pd

df = pd.DataFrame({
    'date': ['2024-01-01', '2024-01-02', '2024-01-03'],
    'sales': [100, 150, 130]
})

df['date'] = pd.to_datetime(df['date'])
df = df.set_index('date')
print(df)
#           sales
# date
# 2024-01-01    100
# 2024-01-02    150
# 2024-01-03    130

df.index      # DatetimeIndex(['2024-01-01', '2024-01-02', '2024-01-03'], dtype='datetime64[ns]')
```

48.2 Time-based slicing

Once your index is a **DatetimeIndex**, you can slice by date strings:

```
df['2024-01']          # all rows in January 2024
df['2024-01-15':'2024-02-15'] # range
df.loc['2024-01-15']    # single date
df['2024']              # all rows in 2024
```

48.3 Date components

Extract parts of the date once it's a **DatetimeIndex**:

```
df.index.year          # 2024
df.index.month         # 1, 2, 3, ...
df.index.day           # 1-31
df.index.dayofweek     # 0=Monday, 6=Sunday
df.index.quarter       # 1-4
df.index.is_month_end  # True/False
```

48.4 Important time series concepts

Concept	Meaning
Trend	Long-term direction (going up or down) in the data
Seasonality	Regular repeating pattern (daily, weekly, yearly)
Cyclical	Irregular repeating pattern (e.g., business cycles), without fixed period
Noise / residual	Random fluctuations after removing trend & seasonality
Stationarity	Statistical properties (mean, variance) don't change over time
Autocorrelation	How correlated a series is with its own past values

WATCH OUT: Time-series ML rules: NEVER shuffle. NEVER use cross-validation that randomly shuffles. Instead, use **TimeSeriesSplit** from sklearn (covered in Part 4 section 37.3) — it always trains on past, validates on future.

49. Time Resampling

Resampling means changing the frequency of your data. You might have hourly data and want daily averages, or daily data and want monthly totals. Pandas does this with the `.resample()` method.

49.1 Downsampling vs upsampling

- **Downsampling:** reduce frequency (hourly → daily). Aggregate the values: mean, sum, max, etc.
- **Upsampling:** increase frequency (monthly → daily). Fill in the gaps: forward-fill, interpolation.

49.2 Resample syntax

```
# Daily data, downsample to monthly mean:
df.resample('M').mean()      # M = month end
df.resample('MS').mean()    # MS = month start
df.resample('Q').sum()      # Q = quarter
df.resample('Y').max()      # Y = year

# Hourly data, downsample to daily:
df.resample('D').mean()     # D = day

# Multiple aggregations at once:
df.resample('M').agg(['mean', 'sum', 'std'])
```

Common frequency strings (offset aliases)

Code	Meaning	Code	Meaning
'D'	Calendar day	'M'	Month end
'B'	Business day	'MS'	Month start
'H'	Hour	'Q'	Quarter end
'T' or 'min'	Minute	'Y' or 'A'	Year end
'S'	Second	'W'	Week

49.3 Upsampling and filling

```
# Monthly to daily; need to fill new dates:
df.resample('D').ffill()    # forward-fill (carry last value forward)
df.resample('D').bfill()    # backward-fill
df.resample('D').interpolate() # linear interpolation
df.resample('D').asfreq()   # leave as NaN
```

NOTE: Common resampling PYQ: 'I have hourly sales data. What does `df.resample("D").sum()` do?' Answer: aggregates the 24 hourly values per day into a single daily total. The result has one row per day.

49.4 Rolling windows

Rolling windows compute statistics over a sliding window of recent values — useful for moving averages, smoothing, and momentum-style features.

```
df['sales'].rolling(window=7).mean()    # 7-day moving average
```

```
df['sales'].rolling(window=30).std()    # 30-day rolling std
df['sales'].rolling(window=7, min_periods=1).mean() # allow shorter windows at start

# Expanding window: window starts at 1 and grows
df['sales'].expanding().mean()          # cumulative mean
```

50. Decomposition: Trend, Seasonality, Residual

Decomposition splits a time series into its three components — trend, seasonality, and residual (noise). Useful for understanding what's driving the data.

50.1 Two decomposition models

Additive model: the components add up. Use when seasonal swings are constant in absolute size, regardless of trend level.

$$y(t) = \text{Trend}(t) + \text{Seasonality}(t) + \text{Residual}(t)$$

Multiplicative model: the components multiply. Use when seasonal swings grow or shrink with the trend (e.g., bigger absolute swings as a business grows).

$$y(t) = \text{Trend}(t) \times \text{Seasonality}(t) \times \text{Residual}(t)$$

50.2 Classical Decomposition

Classical decomposition uses **moving averages** to estimate the trend, then subtracts (or divides) it out and averages over seasons to estimate seasonality.

```
from statsmodels.tsa.seasonal import seasonal_decompose

result = seasonal_decompose(series, model='additive', period=12)
result.trend      # the estimated trend
result.seasonal   # the estimated seasonal component
result.resid      # the residual (noise) after removing both
result.plot()     # the standard 4-panel plot
```

Strengths and weaknesses of classical decomposition

- **Simple and fast.**
- **Assumes seasonality is fixed** (doesn't change shape over time).
- **Trend is missing** for the first and last few observations (because moving averages can't be centered there).
- **Sensitive to outliers.**

50.3 STL Decomposition (Seasonal-Trend decomposition using LOESS)

STL is a more flexible, modern alternative. It uses **LOESS** (locally weighted regression) to estimate the trend and seasonal components, allowing them to evolve over time. It's more robust than classical decomposition.

```
from statsmodels.tsa.seasonal import STL

stl = STL(series, period=12, robust=True)
result = stl.fit()
result.trend
result.seasonal
result.resid
```

Why STL beats classical

- **Seasonality can change over time** (for example, a business's seasonal pattern shifts as it grows).
- **Robust to outliers** when `robust=True`.
- **Handles any seasonal frequency**, not just monthly/quarterly.
- **Trend covers all observations** (no missing values at the ends).

NOTE: Quick comparison:

- **Classical**: simple moving-average based, fixed seasonality, fast but rigid.
- **STL**: LOESS-based, adaptive seasonality, robust to outliers, more flexible.

51. Stationarity & the ADF Test

Stationarity is the most important concept in time series analysis. Many time-series models (ARIMA, Holt-Winters, etc.) **require stationary data** to work. Understanding what stationarity means and how to test for it is critical.

51.1 What is a stationary time series?

A series is **stationary** if its statistical properties don't change over time. Specifically (strict stationarity):

- **Mean is constant** over time (no trend).
- **Variance is constant** over time (no changes in volatility).
- **Autocorrelation** depends only on lag, not on absolute time.

In practice, we usually care about **weak (covariance) stationarity**: just the first two conditions.

51.2 What does NON-stationary look like?

- **Trend**: mean drifts up or down over time. (e.g., a stock price climbing decade after decade.)
- **Seasonality**: mean varies cyclically. (e.g., ice cream sales higher in summer.)
- **Heteroscedasticity**: variance changes over time. (e.g., volatile periods alternating with calm ones.)
- **Random walk**: each value is the previous value plus noise; variance grows with time.

51.3 The Augmented Dickey-Fuller (ADF) Test

ADF is a statistical hypothesis test that checks whether a time series is stationary.

Hypotheses:

- **H_0 (null hypothesis)**: the series is NON-stationary (has a unit root).
- **H_1 (alternative)**: the series IS stationary.

Decision rule:

- If **p-value < 0.05**: reject $H_0 \rightarrow$ series is stationary.
- If **p-value $\geq$ 0.05**: fail to reject $H_0 \rightarrow$ series is non-stationary.

51.4 ADF in statsmodels

```
from statsmodels.tsa.stattools import adfuller

result = adfuller(series)
print('ADF Statistic:', result[0])
print('p-value:', result[1])
print('Critical Values:', result[4])

# Quick decision:
if result[1] < 0.05:
    print('Stationary')
else:
    print('Non-stationary')
```

NOTE: Dec 2025 Q1 PYQ: 'The adfuller() function in statsmodels is used to test:' Answer: **Whether a time series is stationary.** NOT seasonality (that's a different test). NOT normality. NOT heteroscedasticity (that's the Breusch-Pagan or White test).

51.5 Handling NaN values

Most time-series functions in statsmodels (including **adfuller**, **seasonal_decompose**, **STL**) **do not silently impute NaNs**. They will either drop them, raise an error, or return wrong results, depending on the function. **You must handle NaNs before fitting.**

NOTE: Dec 2025 Q2 PYQ: 'What will typically happen if series_with_nans contains NaNs?' Answer: **fit() will raise an error or drop NaNs depending on the estimator; user should handle NaNs beforehand.** Wrong distractors: "statsmodels imputes NaNs silently" (it doesn't); "NaNs treated as zeros" (wrong); "model uses NaNs to estimate variance" (wrong).

52. Making Non-Stationary Data Stationary

If ADF says your series is non-stationary, you transform it until it becomes stationary. Three common techniques:

52.1 Differencing

Subtract each value from the previous one: $y'(t) = y(t) - y(t-1)$. This removes trend. The resulting series shows the CHANGE between consecutive points, not the absolute level.

```
# First-order differencing:
series_diff = series.diff().dropna()

# Second-order (apply twice if first wasn't enough):
series_diff2 = series.diff().diff().dropna()

# Test stationarity again:
adfuller(series_diff)
```

Most non-stationary series become stationary after one or two rounds of differencing. The number of differences needed is called the **order of integration**, denoted **d** in ARIMA(p, d, q) models.

52.2 Seasonal differencing

If the data is seasonal (period s), differencing at lag s removes the seasonal component:

```
# Yearly seasonality on monthly data: difference at lag 12
series_seasonal_diff = series.diff(periods=12).dropna()
```

52.3 Log transformation

If variance is increasing with the level (multiplicative pattern), take the log to stabilize it:

```
import numpy as np
series_log = np.log(series)
# Often combined with differencing afterwards:
series_log_diff = series_log.diff().dropna()
```

52.4 Detrending by regression

Fit a linear (or polynomial) regression of the series against time, then subtract the fitted trend:

```
from sklearn.linear_model import LinearRegression
import numpy as np

t = np.arange(len(series)).reshape(-1, 1)
trend_model = LinearRegression().fit(t, series.values)
trend = trend_model.predict(t)
detrended = series.values - trend
```

52.5 Decision flowchart

Symptom	Try
Linear trend	First-order differencing OR detrend by regression
Quadratic / curved trend	Second-order differencing
Seasonality (period s)	Seasonal differencing at lag s
Variance grows with level	Log transform first, THEN differencing if needed
After all transforms, still non-stationary	Try Box-Cox transform; investigate structural breaks

TIP: Workflow: (1) Plot the series, (2) Run ADF test, (3) If non-stationary, transform (differencing first), (4) Re-run ADF, (5) Repeat until stationary, (6) Now you can fit ARIMA or other time-series models.

53. Self-check Questions & Wrap-up

53.1 Recommender Systems

1. A new movie has just been released. It has a description and cast info but no ratings yet. Which recommender system can recommend it?
2. Compute the cosine similarity between [1, 2, 2] and [2, 1, 2]. Round to 2 decimals.
3. What does SVD do when applied to a user-item matrix?
4. What's the main difference between user-user and item-item collaborative filtering?
5. Why does cosine similarity work well for ratings even when users have different rating scales?

53.2 Time Series

6. What does adfuller() test?
7. Your ADF p-value is 0.001. What's the conclusion?
8. You have hourly data. df.resample('D').mean() does what?
9. What's the difference between classical decomposition and STL?
10. Your series has a clear linear upward trend. What's the simplest fix to make it stationary?
11. Statsmodels' adfuller is given a series with NaN values. What happens?

Answers

1. **Content-based filtering.** CF (user-user, item-item) needs ratings; SVD-based MF needs ratings too. Content-based works from metadata alone.
2. Dot product = $(1)(2) + (2)(1) + (2)(2) = 2 + 2 + 4 = 8$. $\|a\| = \sqrt{1+4+4} = 3$. $\|b\| = \sqrt{4+1+4} = 3$. **Cosine = $8/(3 \times 3) = 8/9 \approx 0.89$.**
3. **Reduces dimensionality** by approximating R as the product of low-rank user and item matrices, indirectly filling missing values.
4. **User-user:** "users like you liked X" (compares rows). **Item-item:** "items similar to ones you liked" (compares columns). Item-item is preferred at scale because items are usually fewer and more stable.
5. Cosine measures the **angle** between vectors, not magnitude. Two users with the same RANKING pattern but different absolute scales (one rates everything 1-3, another 3-5) will have cosine similarity close to 1.
6. **Whether a time series is stationary** (technically: tests for the presence of a unit root).
7. p-value < 0.05, so **reject H_0** . Conclusion: the series IS stationary.
8. **Aggregates the 24 hourly values per day into a single daily mean.** Resulting series has one row per day.
9. **Classical:** moving-average based, fixed seasonality, missing values at boundaries. **STL:** LOESS-based, allows seasonality to evolve, robust to outliers, no boundary issues.

10. **First-order differencing:** `series.diff().dropna()`.

11. **Either raises an error or drops NaNs silently — depends on the function.** You should handle NaNs (drop or impute) BEFORE calling `adfuller`. Don't assume `statsmodels` handles it for you.

Final Wrap-up — The Complete Library

You now have **5 PDFs covering the entire MLP syllabus and every PYQ topic family from the past 11 papers**. Here's the complete index:

Part	Pages	Coverage
Part 1	24	Pandas + sklearn intro + preprocessing
Part 2	24	Feature selection + PCA + Pipelines + Regression
Part 3	27	Classification + Metrics + Ensembles + MLP
Part 4	20	Clustering + CV + Tuning + Transfer Learning + Cheat Sheet
Part 5	(this)	Recommender Systems + Time Series
Total	~110	The full course, exam-focused

Updated 7-day plan with Part 5

Day	Read	Watch (CampusX)	Practice
1	Part 1	Days 23-28 (Pandas)	—
2	Part 1 (skim)	Days 24-29, 35-39	—
3	Part 2 sec 12-16	Days 42-47	2022 Aug PYQ Q1-Q15
4	Part 2 sec 17-22	Days 48-57	2024 Apr 28 PYQ Q1-Q20
5	Part 3	Days 58-66	2025 Apr 13 PYQ (full)
6	Part 4 sec 34-40 + Part 5	KMeans, HAC, GridSearchCV, Recommender, Time Series	2024 Dec 22 PYQ
7	Part 4 cheat sheet	Skim prof's slides	2025 Dec 18 PYQ as mock exam

Now you genuinely have everything. Read methodically, practice the PYQs, trust the work you've done. You've built more solid ML knowledge in a week than many students build in a semester.

Good luck with the exam.